COLLEGE CODE: 8219

COLLEGE NAME: ST.JOSEPH'S COLLEGE OF ENGINEERING AND TECHNOLOGY

DEPARTMENT: ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

DATE: 01.10.2026

CREDIT COURSE: Salesforce Developer - NM Eng

PROJECT TITLE: WhatNext Vision Motors: Shaping the Future of Mobility with Innovation and Excellence

SUBMITTED BY,

TEAM LEAD:

AFRA M - 821923243004

B99B4EE3B6D3A238709493EA84AD9C67

TEAM MEMBERS:

ABINAYA V - 821923243003

01437221EFBA23EA9BD42C4D29481252

JELINA G - 821923243022

B020E11899693CEB6AA4A7EE418D649E

SANTHIYA P - 821923243045

B00F4F71D14695DBC9149A004FBE0A24

WhatNext Vision Motors: Shaping the Future of Mobility with Innovation and Excellence

Use Case :

WhatsNext Vision Motors, a pioneering force in the automotive industry, is dedicated to transforming the mobility sector with innovative technology and solutions that prioritize customer needs. The company has embarked on an ambitious Salesforce project with the core objective of enhancing the customer experience and streamlining its operational processes.

At the heart of this project is the improvement of the customer ordering process. The system is designed to automatically suggest the nearest dealer location to customers based on their address. This feature is intended to significantly enhance the convenience and efficiency of the ordering experience, making it more customer-friendly and reducing the time and effort required from the customer's end.

The project also addresses a common issue in the automotive industry: stock availability. The system includes a mechanism that prevents customers from placing orders for vehicles that are out of stock. This proactive approach ensures that customers can only create orders for vehicles that are currently available, thus avoiding potential confusion and disappointment that may arise from stock unavailability. This feature not only enhances customer satisfaction but also improves the accuracy of the company's order fulfillment process.

Furthermore, the project incorporates a scheduled process for updating the status of bulk order records. This automated process is designed to update the order status based on stock availability. If a vehicle is out of stock at the time of order placement, the system will update the order status to 'Pending.' On the other hand, if the vehicle is in stock, the system will update the status to 'Confirmed.' This ensures that all orders are accurately reflected in terms of their fulfillment status, providing clear and transparent communication to customers regarding the status of their orders.

The implementation of this Salesforce project at WhatsNext Vision Motors is expected to yield several benefits. It aims to create a more efficient ordering system that reduces the potential for errors and improves the overall service provided to customers. By streamlining the ordering process and ensuring accurate stock availability, the company can enhance customer satisfaction and loyalty.

Moreover, the project is expected to contribute to operational efficiency by reducing the administrative burden on staff. By automating certain processes, employees can focus on more strategic tasks that require human intervention and expertise. This not only improves the overall

productivity of the company but also allows for a more agile response to market demands and customer needs.

Requirements :

1. **Salesforce CRM Implementation**
    - Store and manage vehicle details, stock availability, and dealer information in Salesforce.
    - Track customer orders, test drives, and service requests efficiently.
    - Automate workflows to assign orders to the nearest dealer based on customer location.

2. **Process Automation**
    - Prevent order placement if the vehicle is out of stock.
    - Auto-assign orders to the nearest dealer based on the customer's location.
    - Send automated email reminders for scheduled test drives.

3. **Apex and Trigger**

    - Implement Apex triggers to enforce business rules such as stock validation and automatic dealer assignment.
    - Use trigger handlers to follow best practices and ensure modularity and maintainability.

4. **Batch Jobs**

    - Develop a batch Apex job to periodically check vehicle stock levels and update availability.
    - Send scheduled email notifications for stock replenishment and order processing.

What you'll learn :

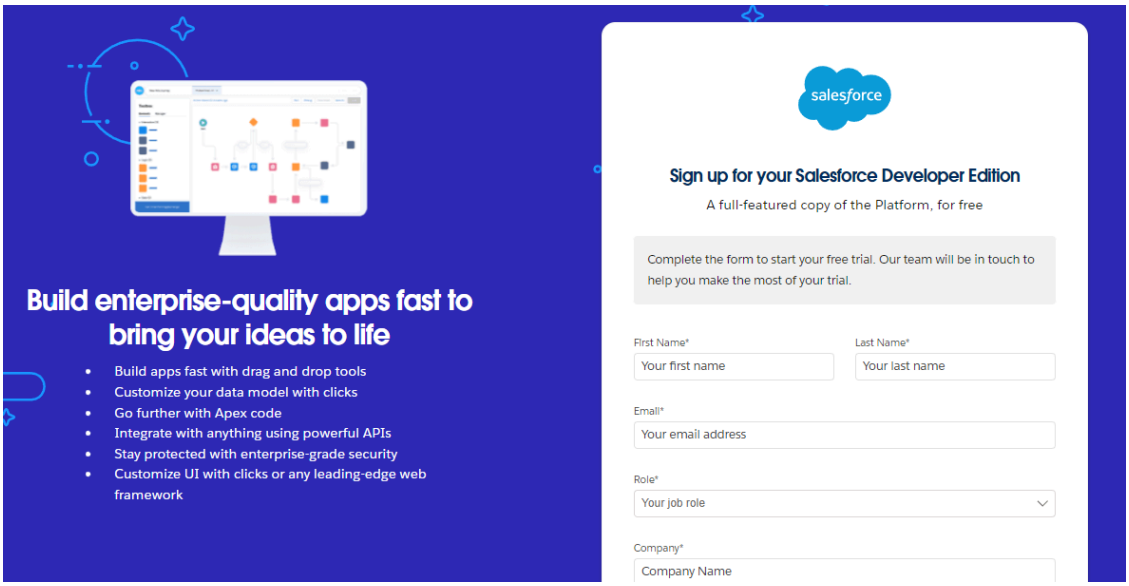
1. Data Modelling
2. Fields and Relationships
3. Lightning App Builder
4. Record Triggered Flows
5. Apex and Apex Triggers
6. Batch Apex
7. Scheduled Apex

MileStone 1 :

Activity 1: Creating Developer Account

Creating a developer org in salesforce.

1. Go to https://developer.salesforce.com/signup
2. On the sign up form, enter the following details :



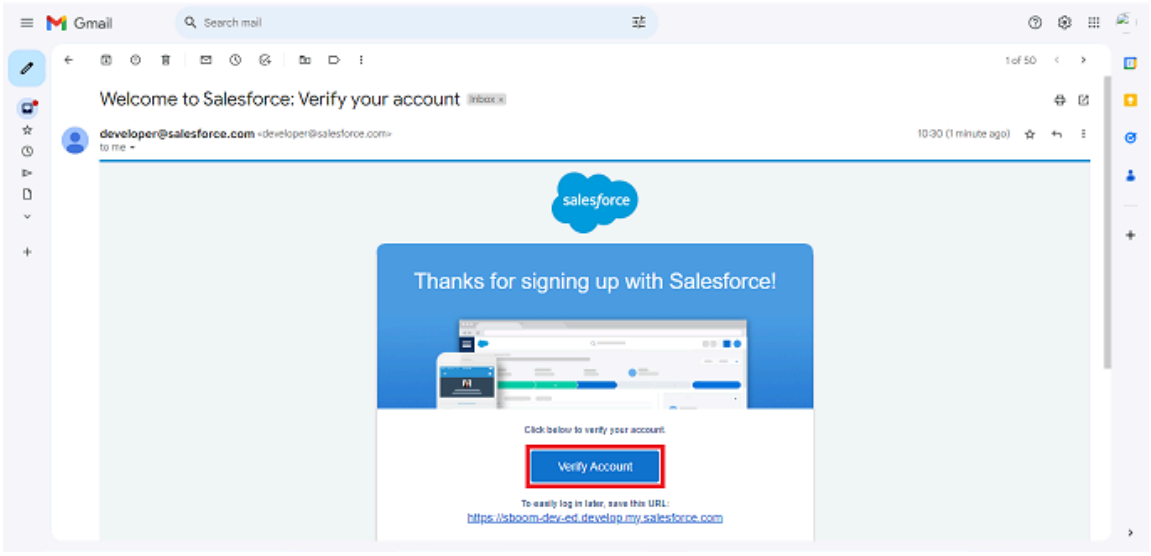
1) First name & Last name
2) Email
3) Role : Developer
4) Company : College Name
5) County : India
6) Postal Code : pin code
7) Username : should be a combination of your name and company

This need not be an actual email id, you can give anything in the format : username@organization.com

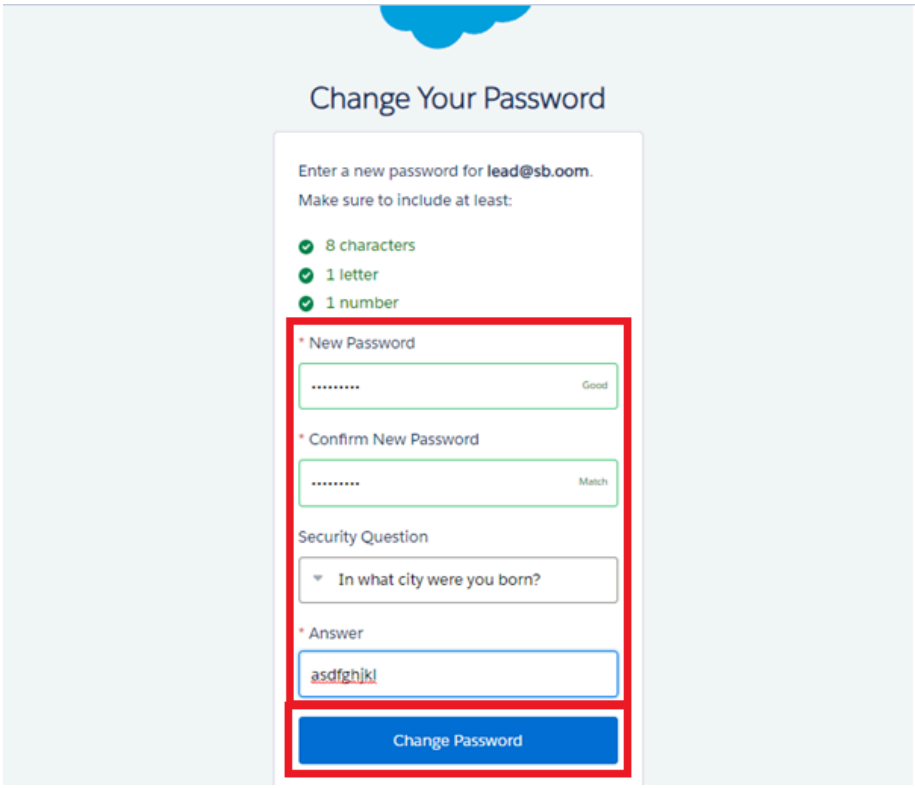
Click on sign me up after filling these.

Activity 2: Account Activation

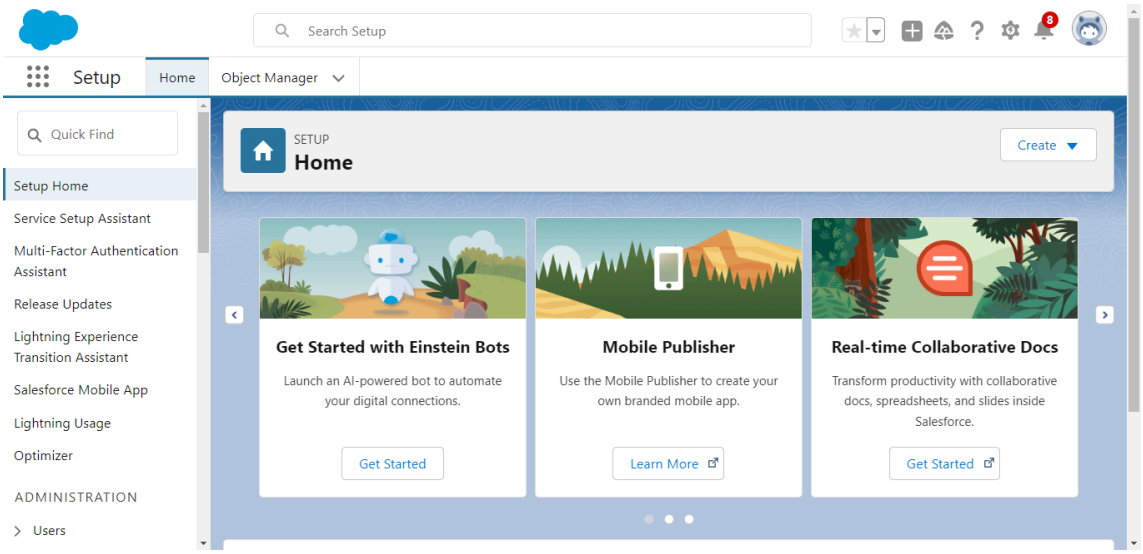
1. Go to the inbox of the email that you used while signing up. Click on the verify account to activate your account. The email may take 5-10mins.



2. Click on Verify Account
3. Give a password and answer a security question and click on change password.



4. Then you will redirect to your salesforce setup page.

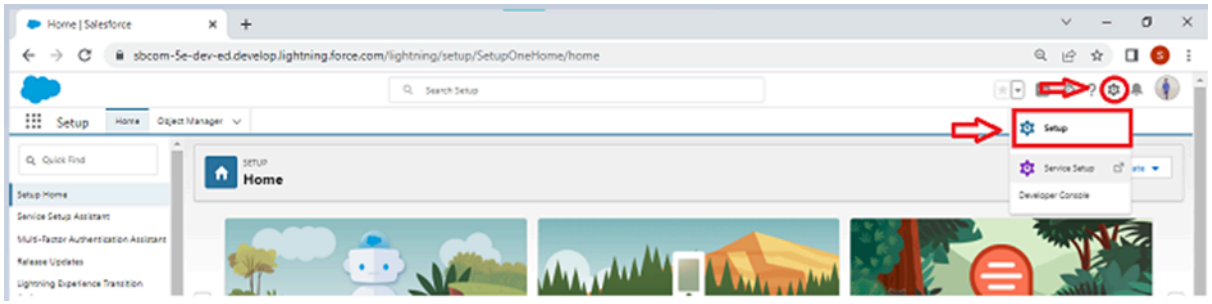

Search Setup
Setup
Home
Object Manager
Quick Find
Setup Home
Service Setup Assistant
Multi-Factor Authentication Assistant
Release Updates
Lightning Experience Transition Assistant
Salesforce Mobile App
Lightning Usage
Optimizer
ADMINISTRATION
Users
SETUP
Home
Create
Get Started with Einstein Bots
Launch an AI-powered bot to automate your digital connections.
Get Started
Mobile Publisher
Use the Mobile Publisher to create your own branded mobile app.
Learn More
Real-time Collaborative Docs
Transform productivity with collaborative docs, spreadsheets, and slides inside Salesforce.
Get Started

MileStone 2 :

Objects & Relationships

Object Name	Purpose	Relationships
Vehicle__c	Stores vehicle details	Related to Dealer & Orders
Vehicle_Dealer__c	Stores authorized dealer info	Related to Orders
Vehicle_Customer__c	Stores customer details	Related to Orders & Test Drives
Vehicle_Order__c	Tracks vehicle purchases	Related to Customer & Vehicle
Vehicle_Test_Drive__c	Tracks test drive bookings	Related to Customer & Vehicle
Vehicle_Service_Request__c	Tracks vehicle servicing requests	Related to Customer & Vehicle

To Navigate to Setup page:
Click on gear icon → click setup.



Activity 1: Create Vehicle Object

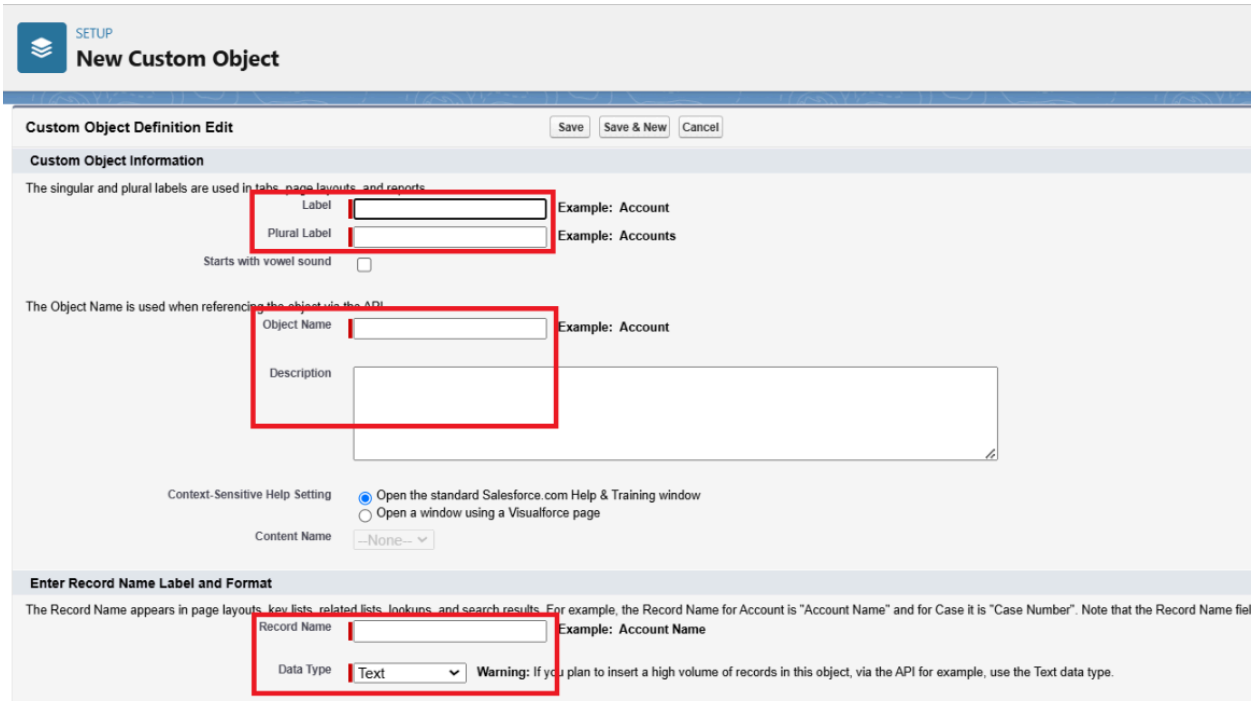
The purpose of creating a Vehicle custom object is to have a clear picture of the vehicle details.

To create an object:

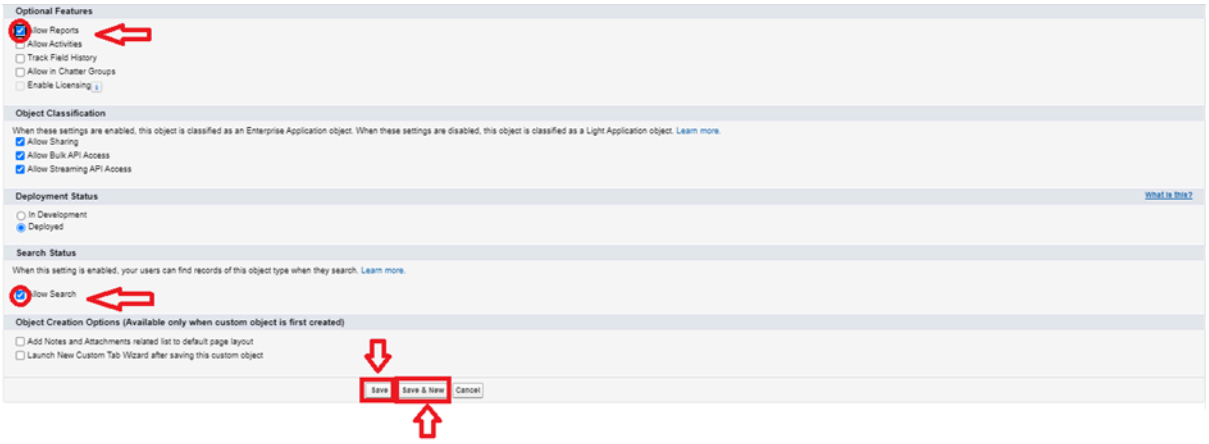
1. From the setup page → Click on Object Manager → Click on Create → Click on Custom Object.



1) Enter the label name→ Vehicle
2) Plural label name→ Vehicles
3) Enter Record Name Label and Format
    - Record Name → Vehicle Name
    - Data Type → Text



2. Click on Allow reports,
3. Allow search → **Save.**



Activity 2: Create Dealer Object

The purpose of creating a Dealer object is to have detailed information about Dealer.

To create an object:

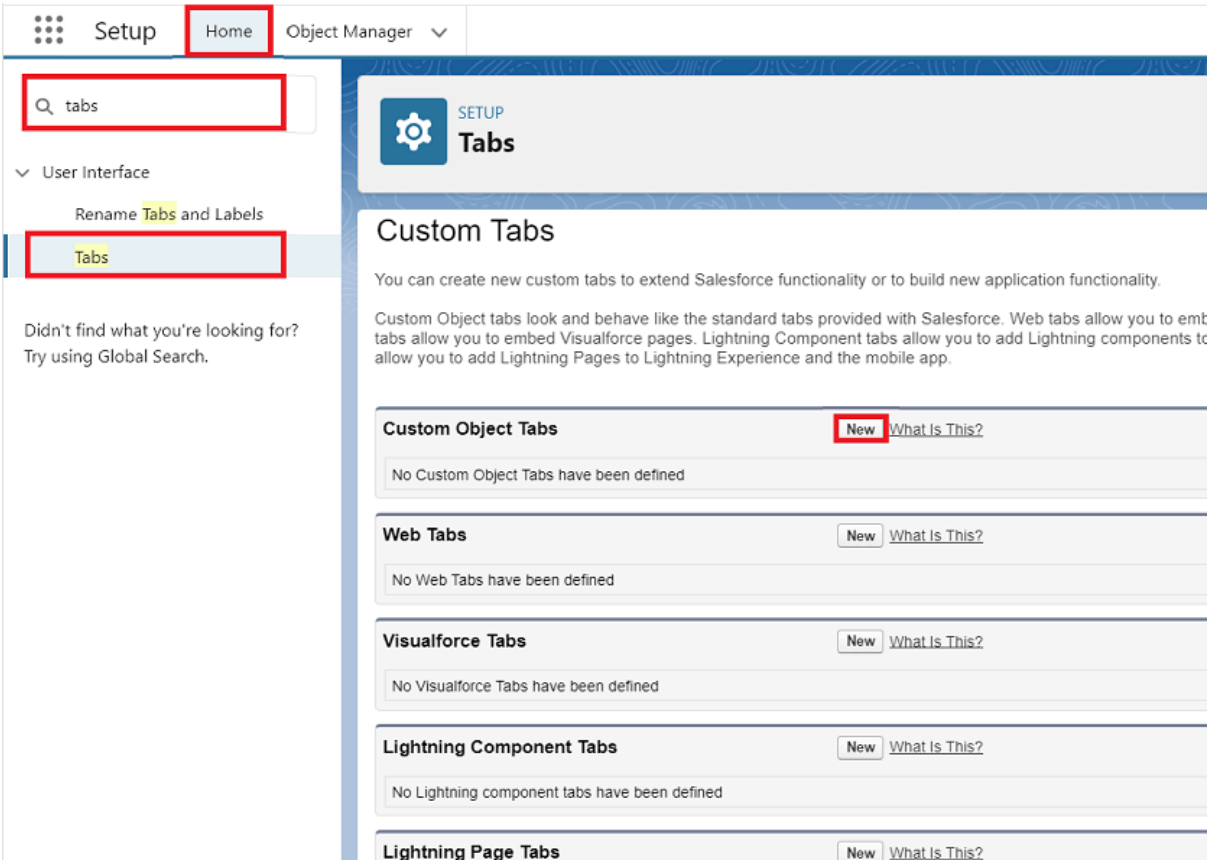
1. From the setup page → Click on Object Manager → Click on Create → Click on Custom Object.
    1) Enter the label name→ Vehicle Dealer
    2) Plural label name→ Vehicle Dealers
    3) Enter Record Name Label and Format
        - Record Name → Dealer Name
        - Data Type → Text
2. Click on Allow reports,
3. Allow search → **Save**

Activity 3: Creating remaining objects

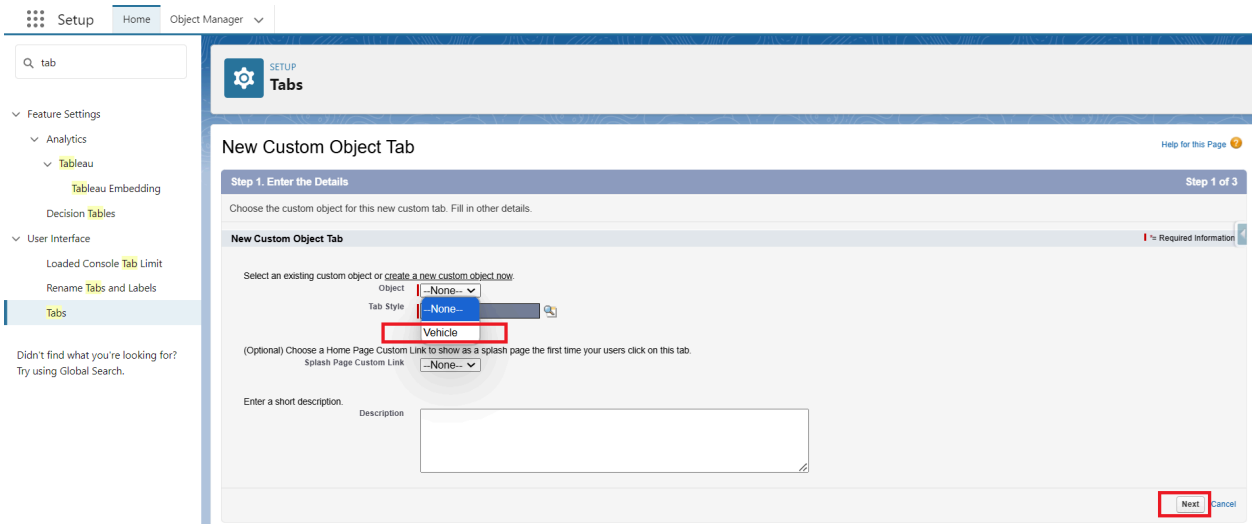
Milestone 3 - Tabs

Activity 1: Creating a Custom Tab(Vehicle)

1. Go to setup page → type Tabs in Quick Find bar → click on tabs → New (under custom object tab)



2. Select Object(Vehicle) → Select any tab style → Next (Add to profiles page) keep it as default → Next (Add to Custom App) keep it as default → Save.



Activity 2:

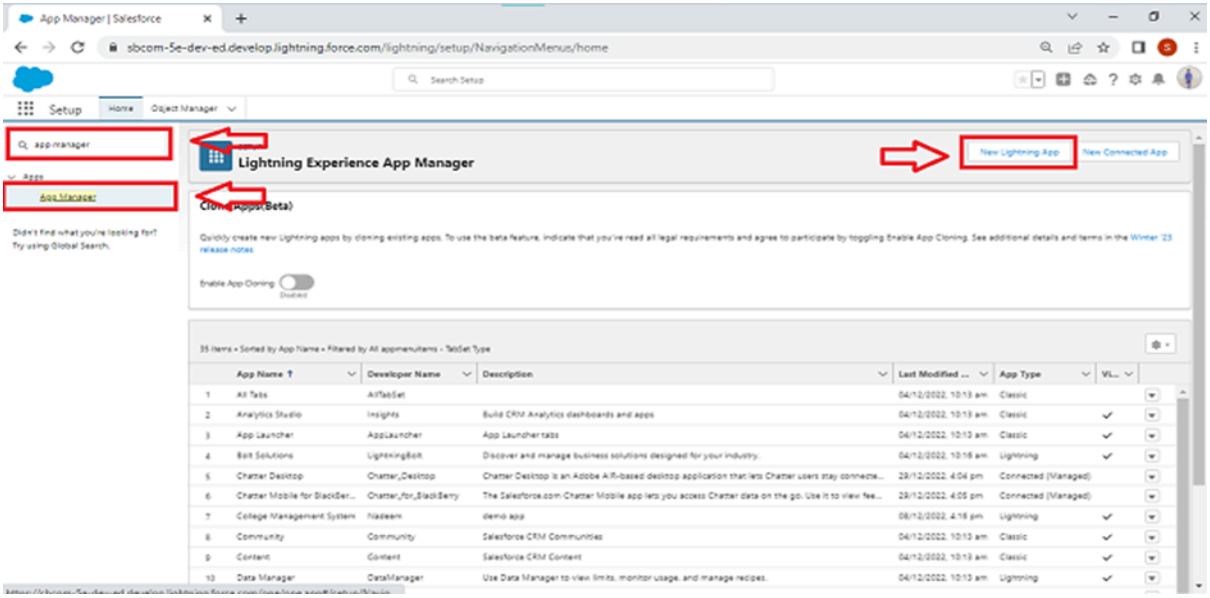
Now create tabs for Other objects as well which we have created above Objects.

Milestone 4- The Lightning App:

Activity 1: Create a Lightning App

To create a lightning app page:

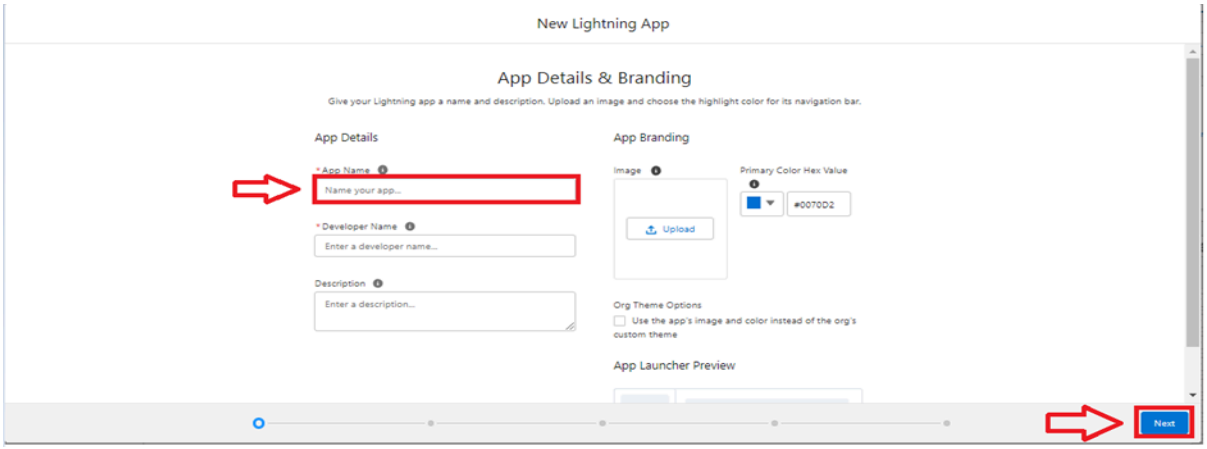
1. Go to setup page → search “app manager” in quick find → select “app manager” → click on New lightning App.



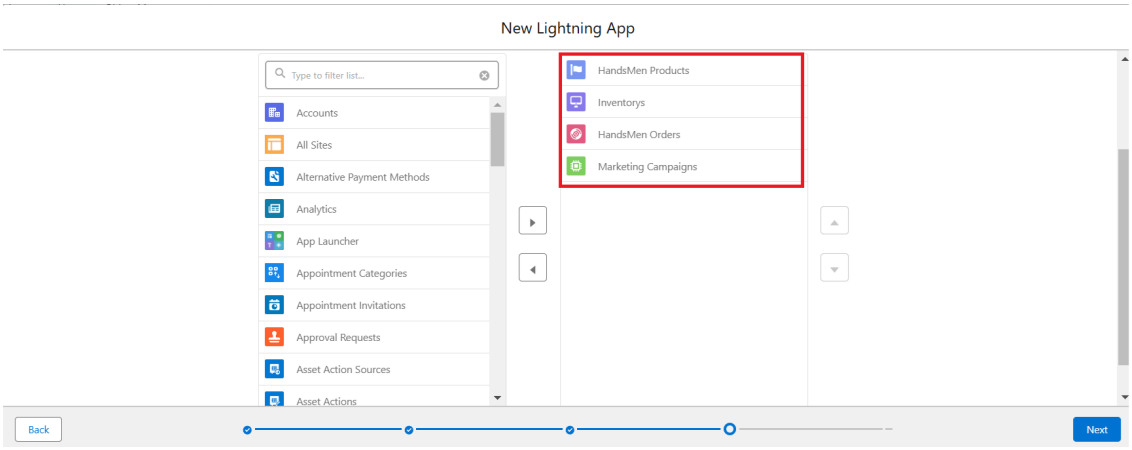
2. Fill the app name in app details and branding as follow
   App Name : WhatNext Vision Motors

Developer Name : this will auto populated
Description : Give a meaningful description
Image : optional (if you want to give any image you can otherwise not mandatory)
Primary color hex value : keep this default

3. Then click Next → (App option page) keep it as default → Next → (Utility Items) keep it as default → Next.



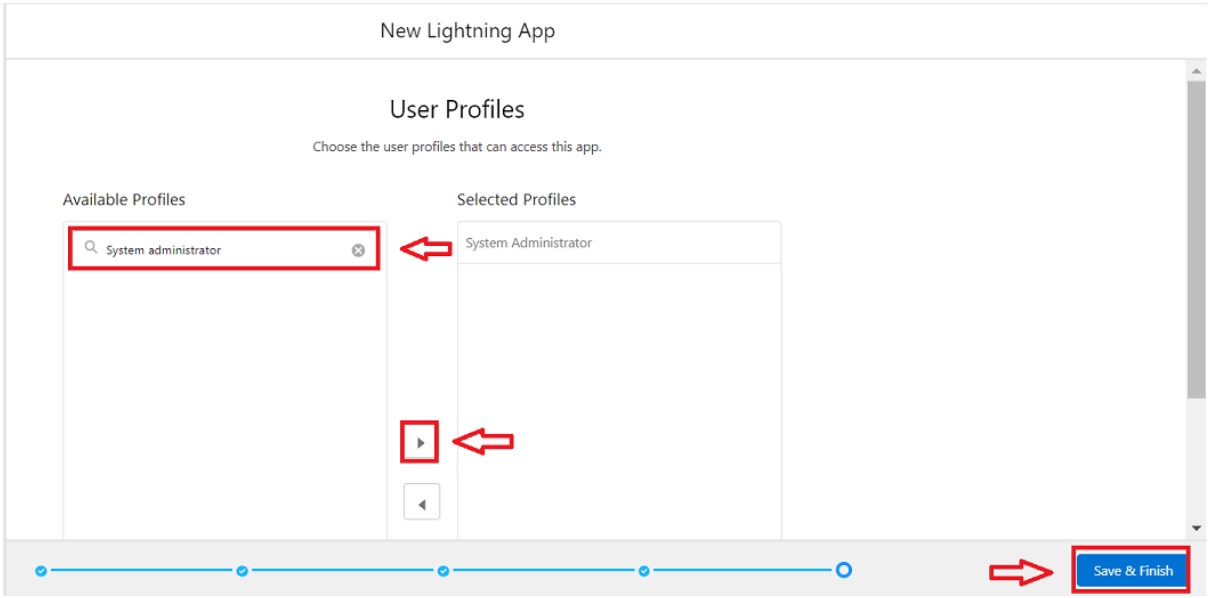
4. To Add Navigation Items:



Search the items in the search bar(Vehicle, Dealer, Customer, Order, Test Drive, Service Request, Reports, Dashboard) from the search bar and move it using the arrow button → Next.

Note: select the custom object which we have created in the previous activity.

5. To Add User Profiles:



Search profiles (System administrator) in the search bar → click on the arrow button → save & finish.

Milestone 5 : Fields & Relationships

Key Fields for Each Object

1. Vehicle__c (Custom Object)

- **Vehicle Name**(Text)
- **Vehicle_Model__c** (Picklist: Sedan, SUV, EV, etc.)
- **Stock_Quantity__c** (Number)
- **Price__c** (Currency)
- **Dealer__c** (Lookup to Dealer__c)
- **Status__c** (Picklist: Available, Out of Stock, Discontinued)

2.Vehicle_ Dealer__c (Custom Object)

- **Dealer_Name__c** (Text)
- **Dealer_Location__c** (Text)
- **Dealer_Code__c** (Auto Number)
- **Phone__c** (Phone)
- **Email__c** (Email)

3. Vehicle_Order__c (Custom Object)

- **Vehicle_Customer__c** (Lookup to Vehicle_Customer__c)
- **Vehicle__c** (Lookup to Vehicle__c)
- **Order_Date__c** (Date)
- **Status__c** (Picklist: Pending, Confirmed, Delivered, Canceled)

4. Vehicle_Customer__c (Custom Object)

- **Customer_Name__c** (Text)
- **Email__c** (Email)
- **Phone__c** (Phone)
- **Address__c** (Text)
- **Preferred_Vehicle_Type__c** (Picklist: Sedan, SUV, EV, etc.)

5. Vehicle_Test_Drive__c (Custom Object)

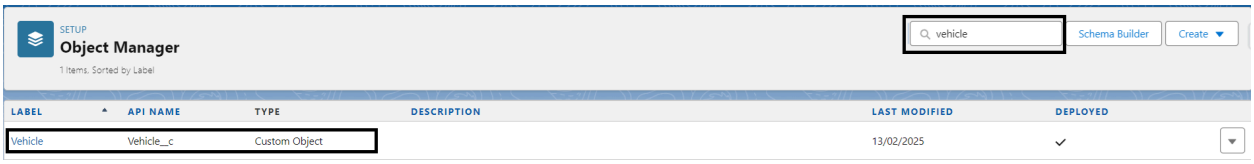
- **Vehicle_Customer__c** (Lookup to Vehicle_Customer__c)
- **Vehicle__c** (Lookup to Vehicle__c)
- **Test_Drive_Date__c** (Date)
- **Status__c** (Picklist: Scheduled, Completed, Canceled)

6. Vehicle_Service_Request__c (Custom Object)

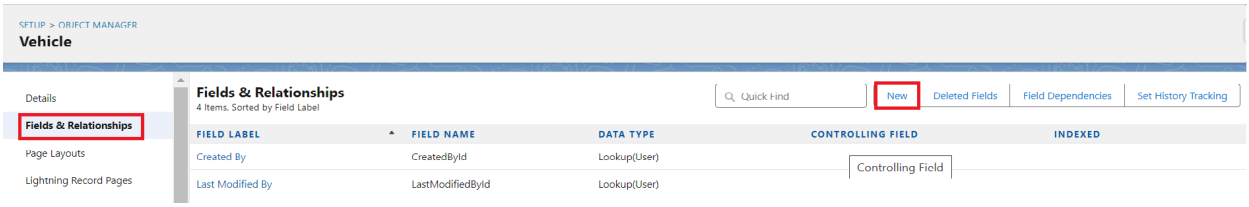
- **Vehicle_Customer__c** (Lookup to Vehicle_Customer__c)
- **Vehicle__c** (Lookup to Vehicle__c)
- **Service_Date__c** (Date)
- **Issue_Description__c** (Text)
- **Status__c** (Picklist: Requested, In Progress, Completed)

Activity 1: Creating a Field in Vehicle Object

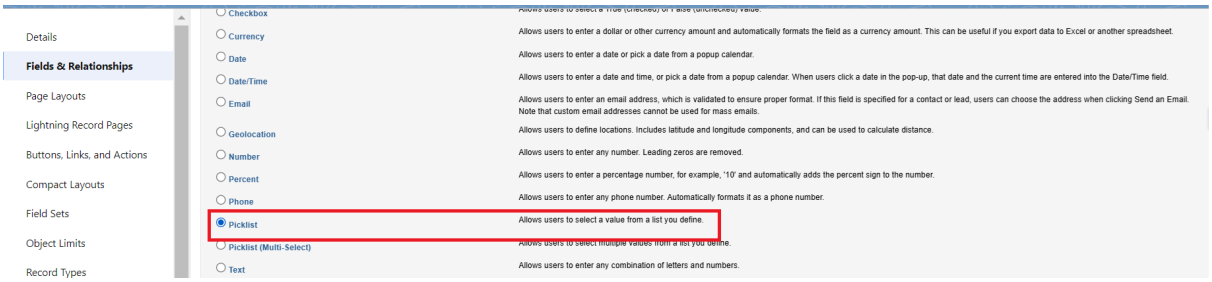
1. Go to setup → click on Object Manager → type object name(Vehicle) in quick find bar→ click on the object.



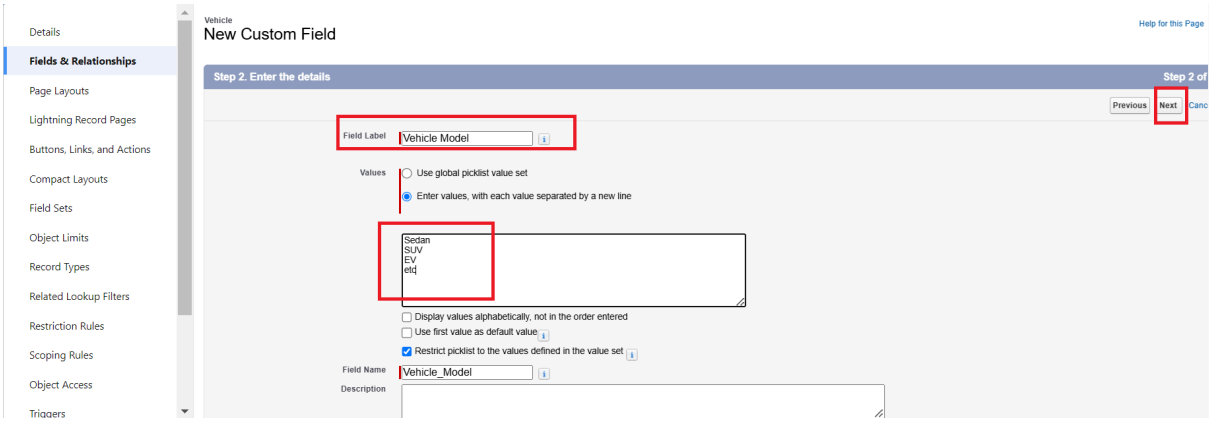
2. Now click on “Fields & Relationships” → New



3. Select Data type as “PickList”.



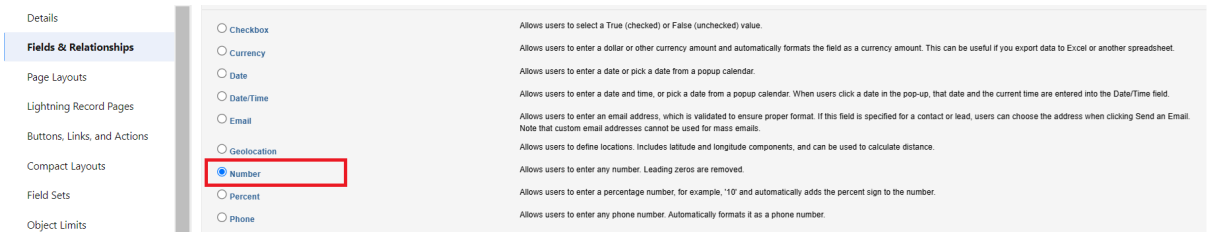
4. Click on Next



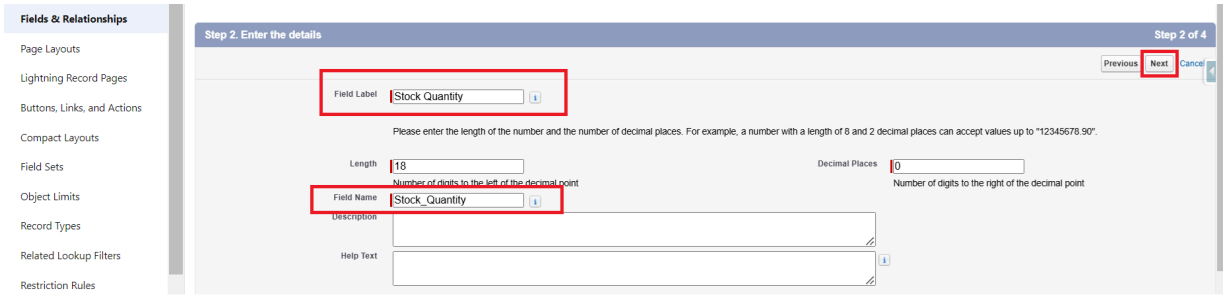
5. Fill the above as following:
   - Field Label: Vehicle Modal
   - Fill PickList Values
   - Click on Next → Next → Save and new.

Activity 2 : Creating Stock Quantity field in vehicle object

1. Repeat the steps 1 & 2 from the activity 1
2. Select Data type as “Number”.



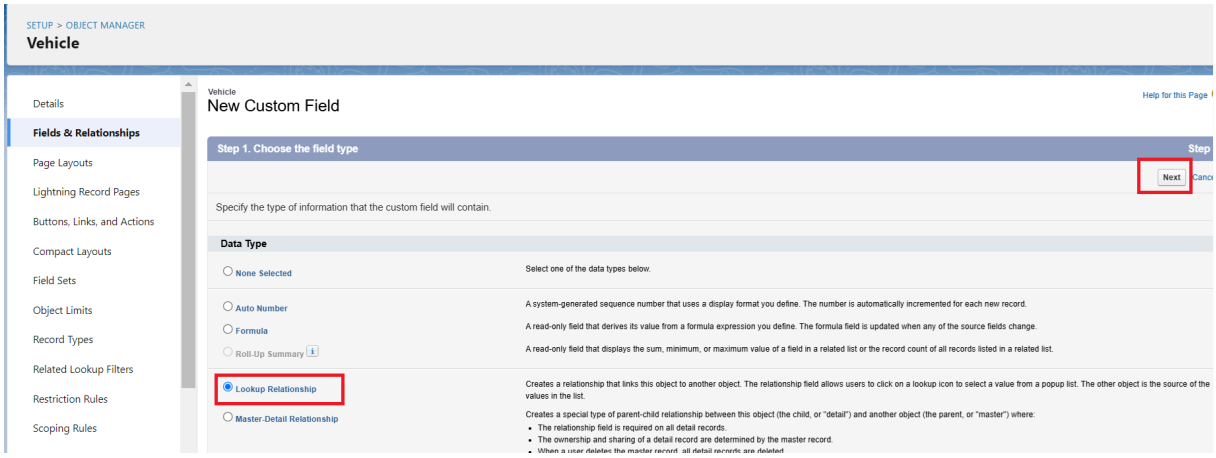
3. Click on Next



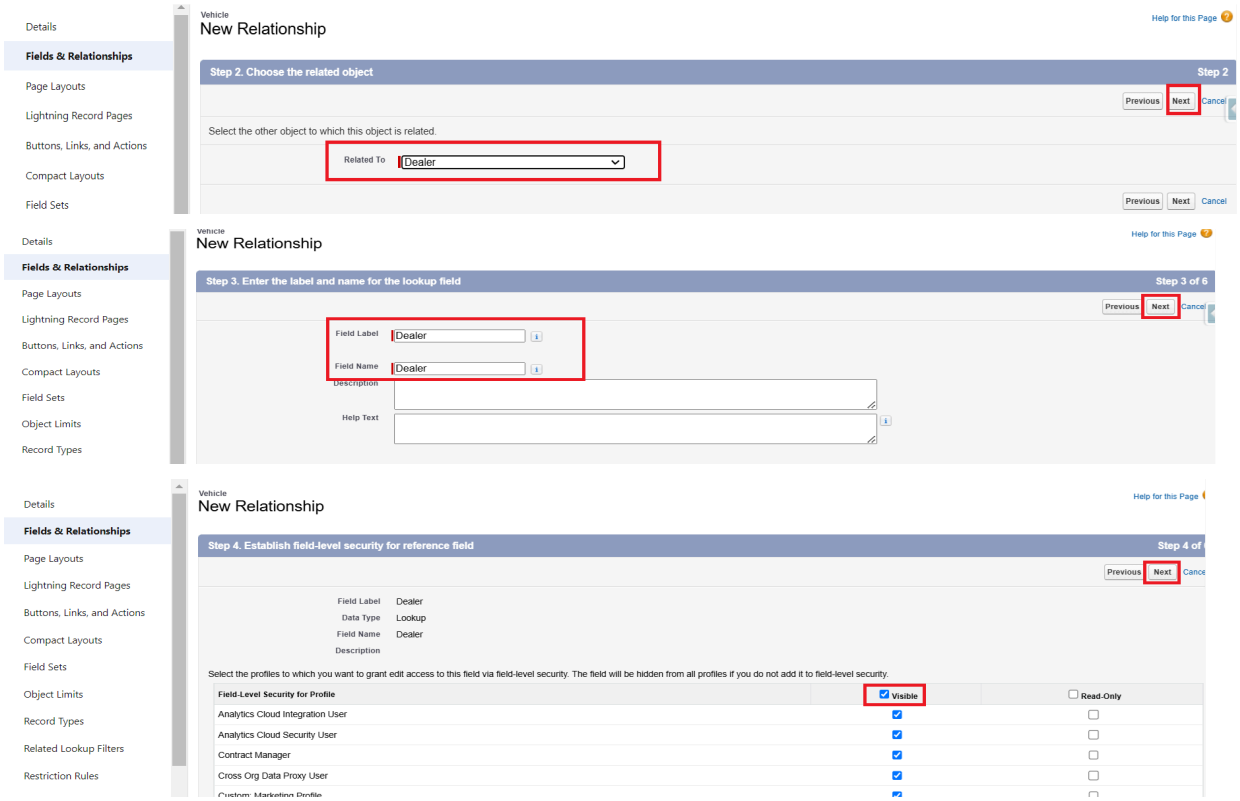
4. Fill the above as following:
   - Field Label: Stock Quality
   - Click on Next → Next → Save and new.

Activity 3 : Creating a Lookup RelationShip in Vehicle Object to Dealer Object.

1. Select DataType as Lookup Relationship.



2. Click Next
3. Select Dealer Object Form Related Filed.



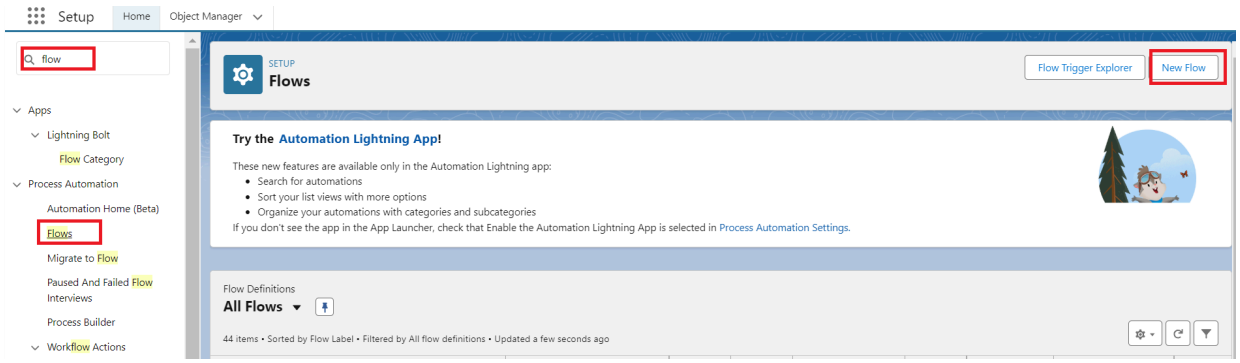
4. Click Next and Save.

Activity 4 : Creating Remaining Objects and Fields.

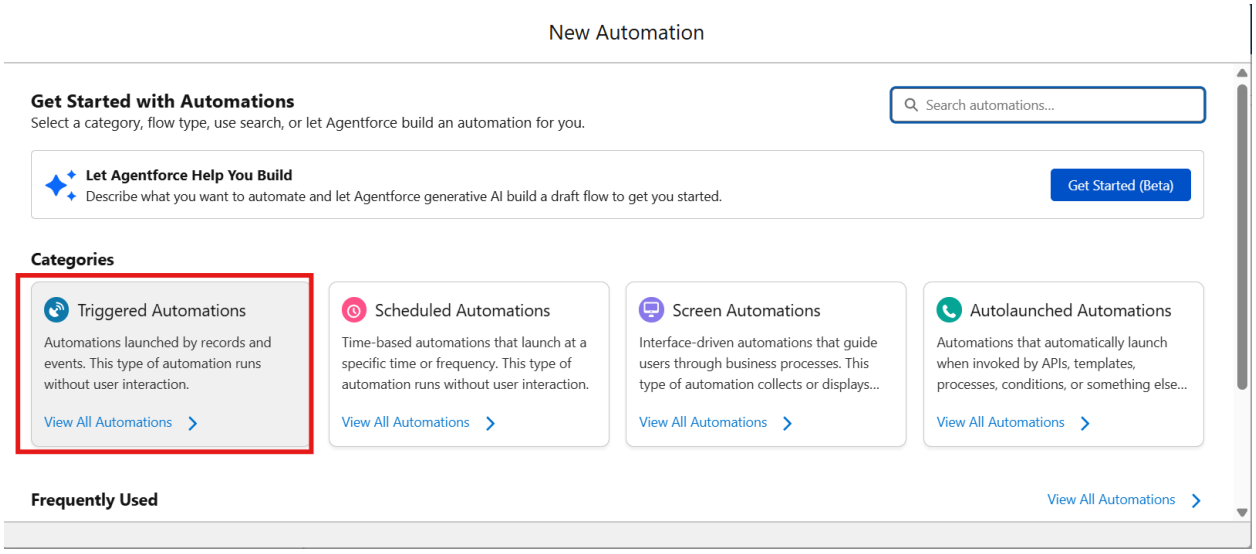
MileStone 6 :

Flow 1 : Creating a record triggered flow to assign nearest dealer to the customer's location

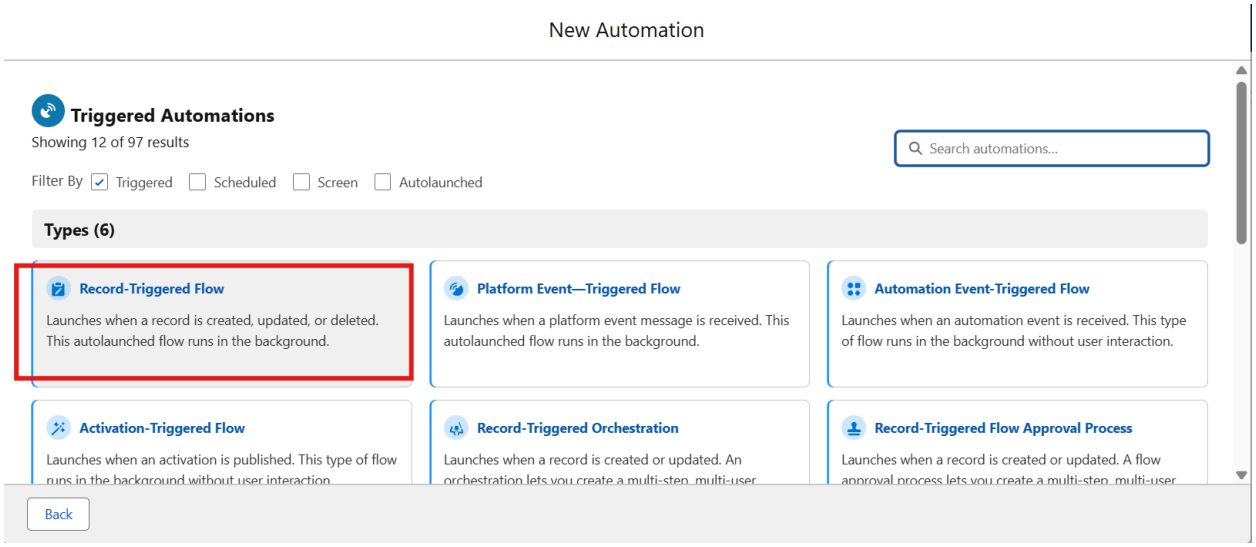
Step 1 : In Quick Find, type Flows and click on Flows. Click New Flow.



Step 2 : Select Triggered Automation From Flow.



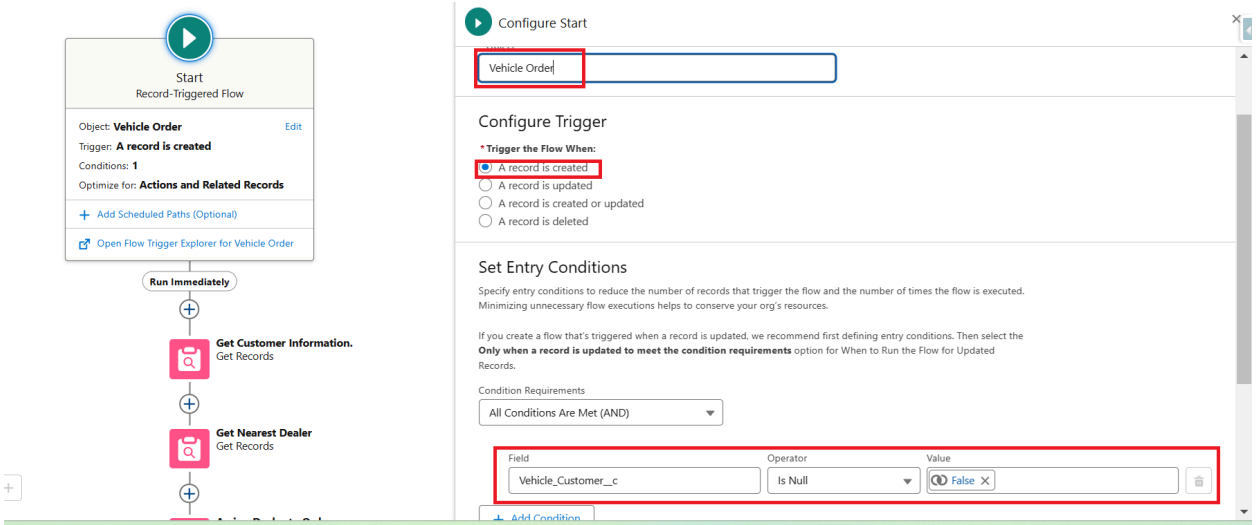
Step 3 : Select Record-Triggered Flow



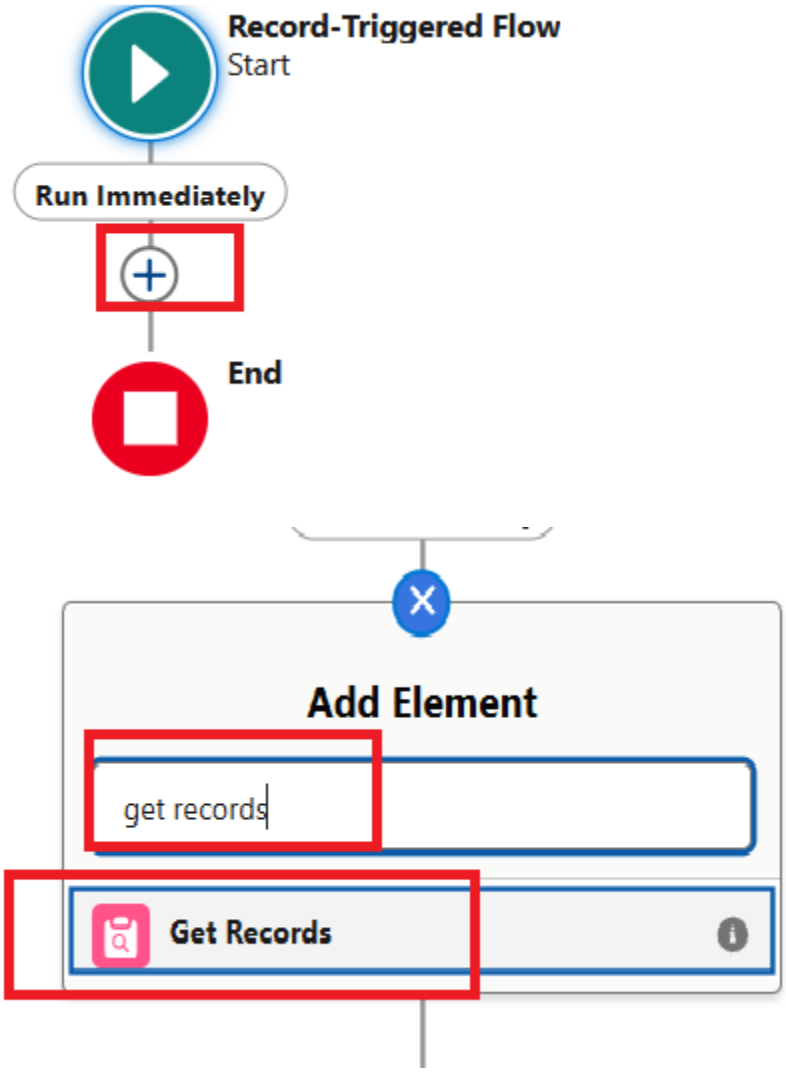
Step 4 : Select Vehicle Order Object

Trigger the Flow When : Select Record is Created.

Set Entry Condition :
All Conditions Are Met (AND)
Filed : Status__c
Operator : Equals
Value : Pending

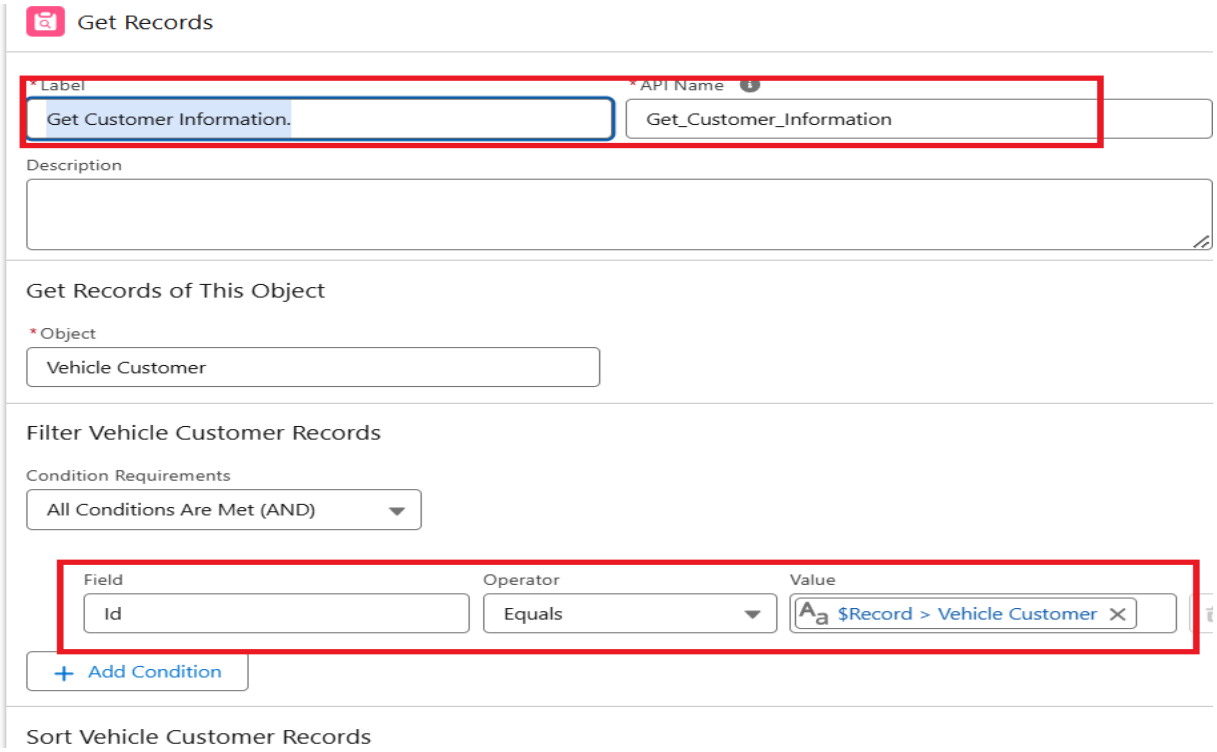


Step 5 : Click + → Select **Get Records**



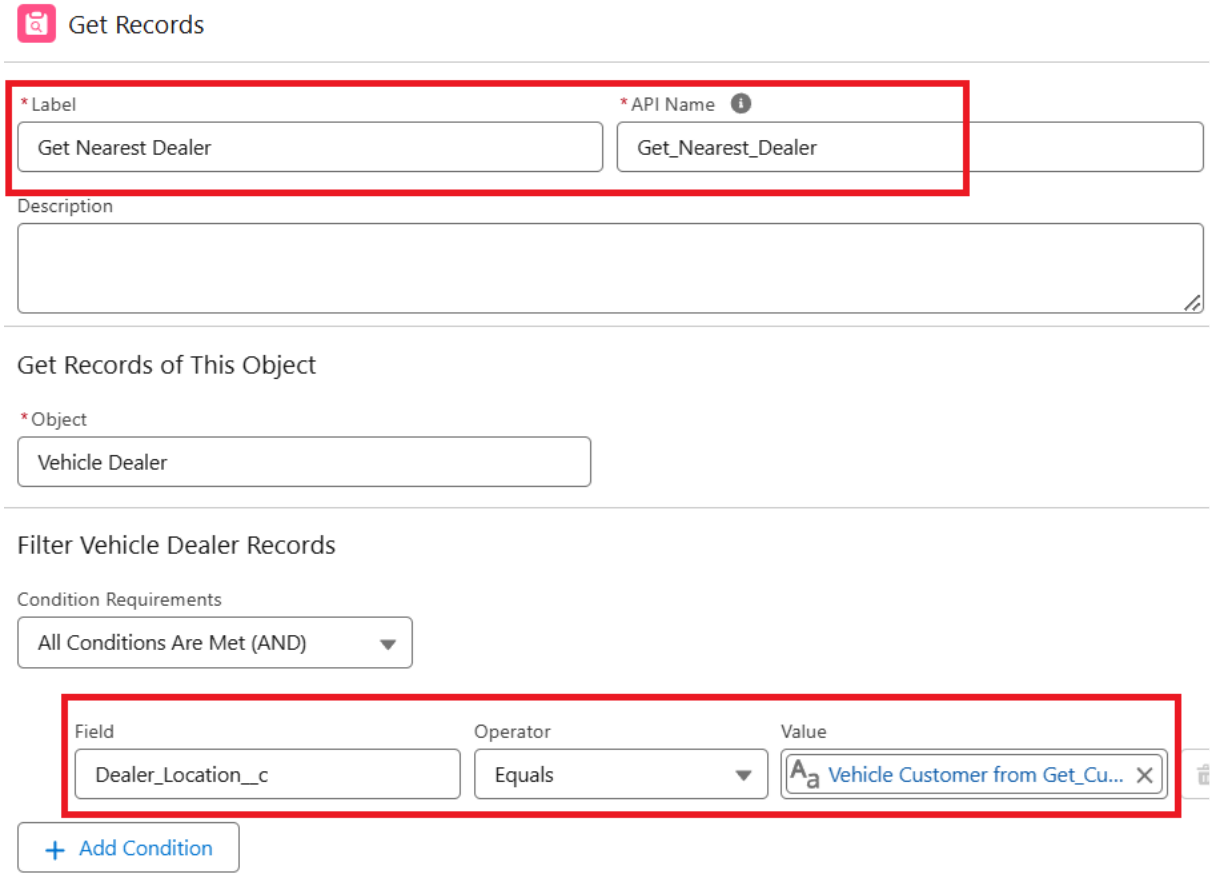
Step 6 : **Label :** Get Customer Information

Object : Vehicle Customer
Condition :
Field : Id
Operator : Equals
Value : {!$Record.Vehicle_Customer__c}



Step 7 : Click + → Select **Get Records**

Label : Get Nearest Dealer
Object : Vehicle Dealer
Condition :
Field : Dealer_Location__c
Operator : Equals
Value : {!Get_Customer_Information.Address__c}

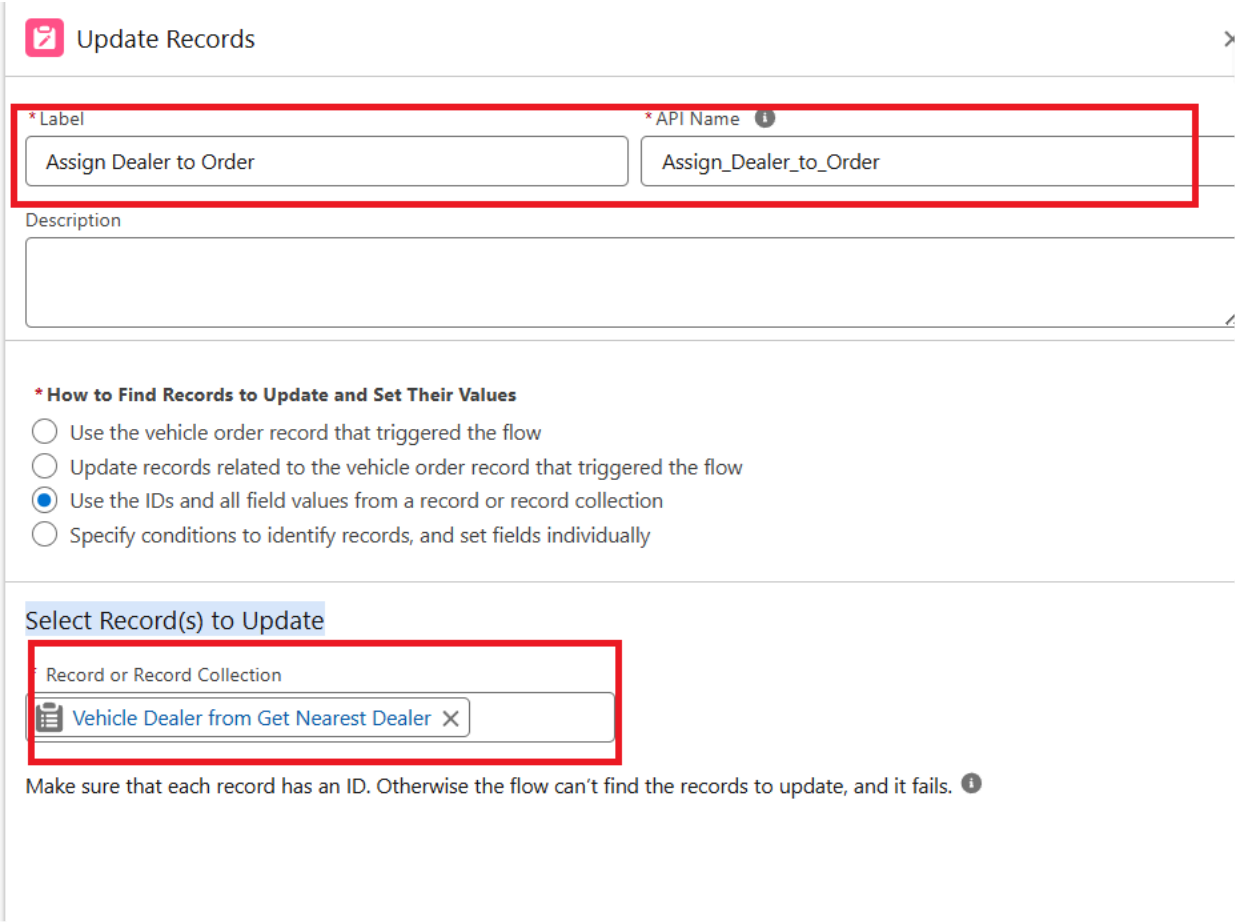


Step 8 : Click + → Select **Update Records**

Label : Assign Dealer to Order

*How to Find Records to Update and Set Their Values : Use the IDs and all field values from a record or record collection

Select Record(s) to Update : {!Get_Nearest_Dealer}

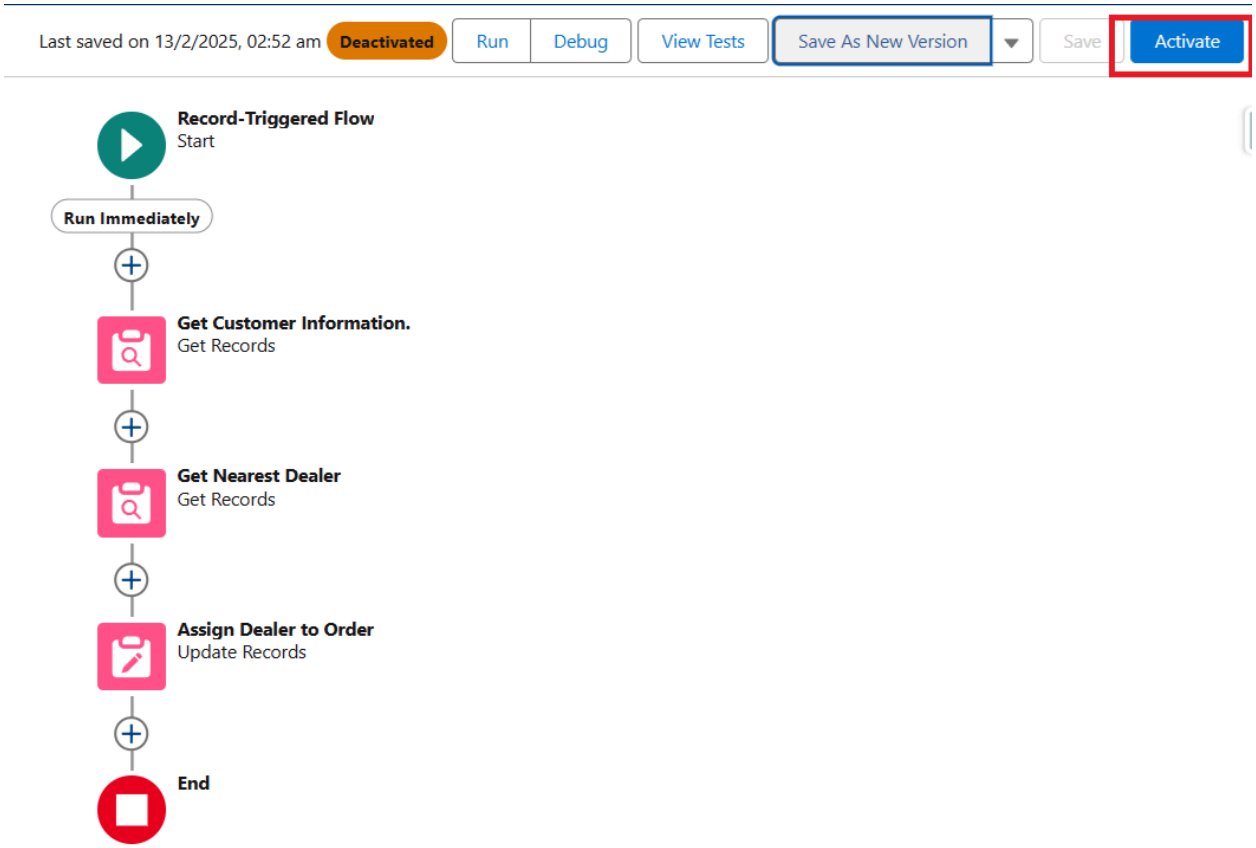


Step 9 : Click Save and Give label Name and Activate Flow.

Label Name : Auto Assign Dealer

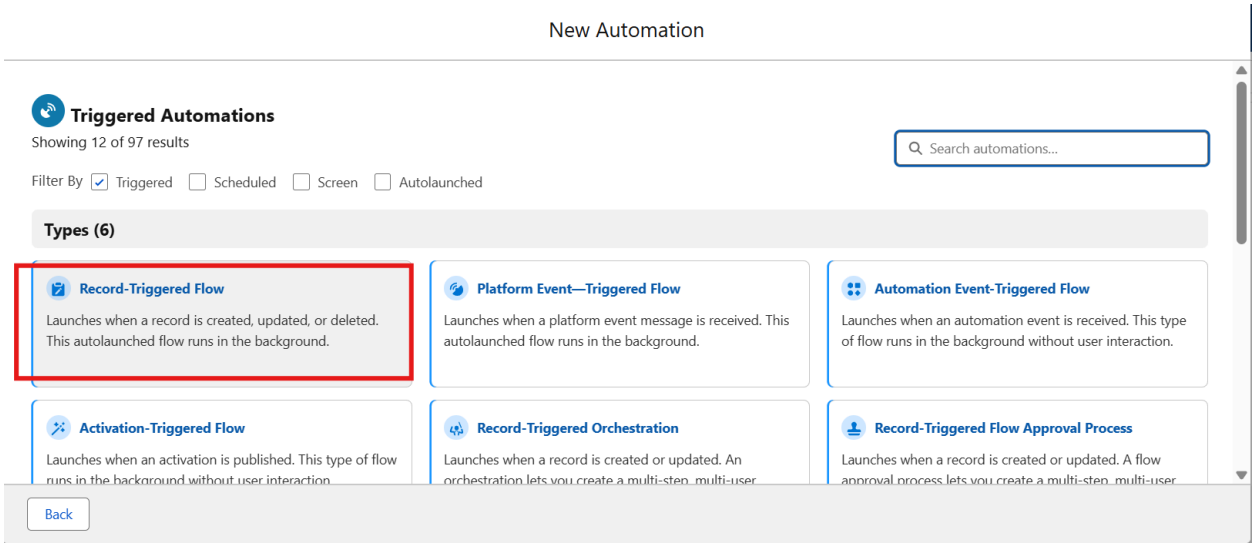


Step 10 : Activate Flow



Flow 2 : Creating record triggered flow to send an email to the customer reminding about the test drive.

Step 1 : Select Record-Triggered Flow



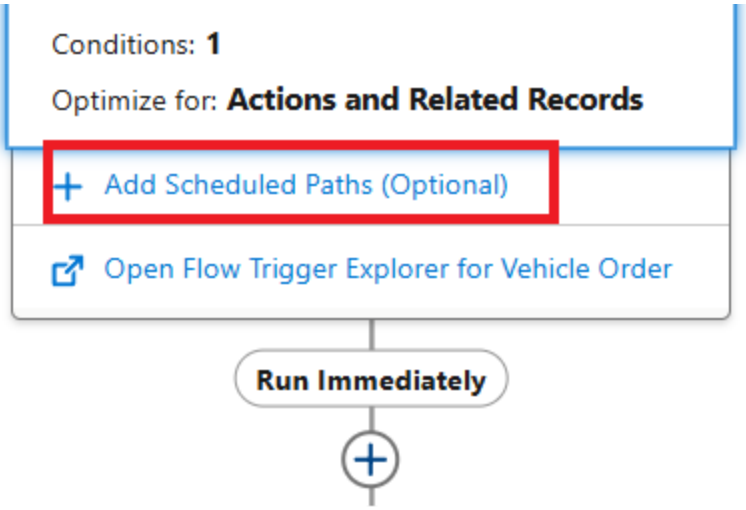
Step 2 : Select Vehicle Test Drive Object

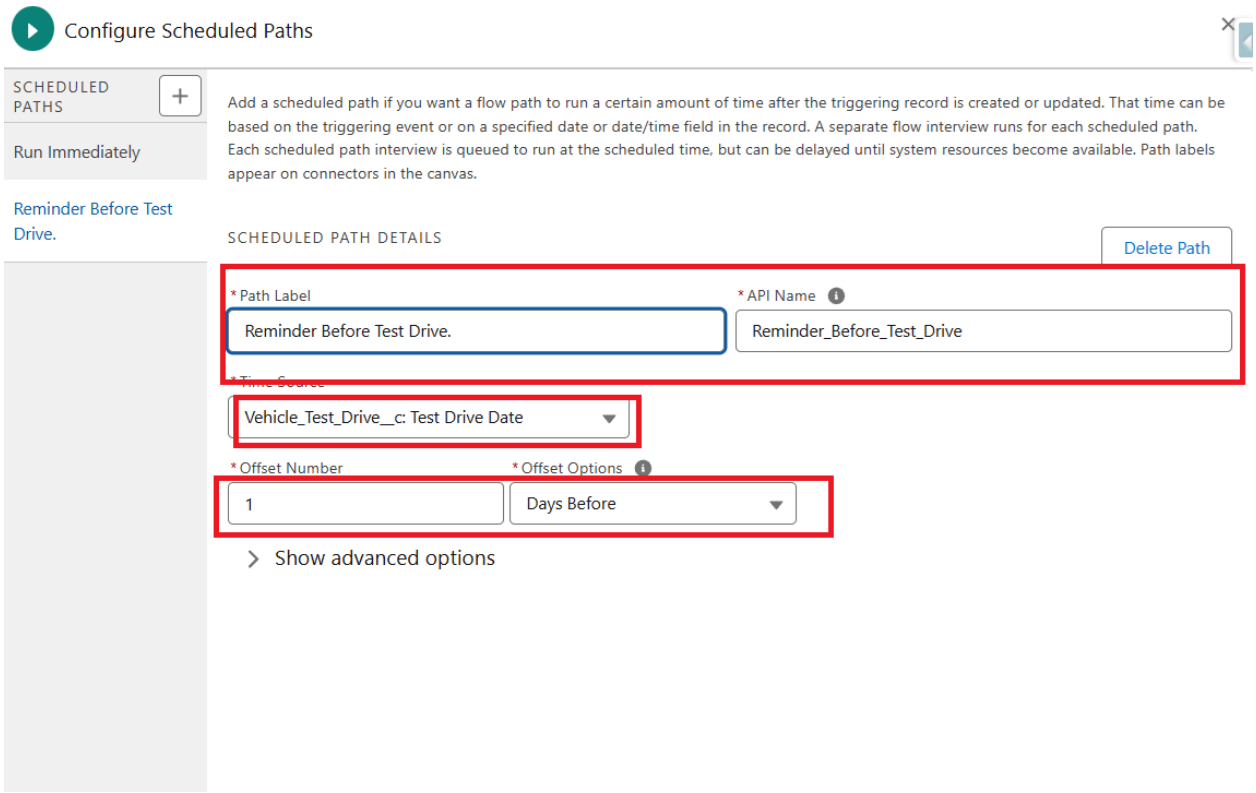
Trigger the Flow When : A record is created or updated

Set Entry Condition :
All Conditions Are Met (AND)
Filed : Status__c
Operator : Equals
Value : Scheduled

Step 3 : Click + Add Scheduled Paths (below the trigger).

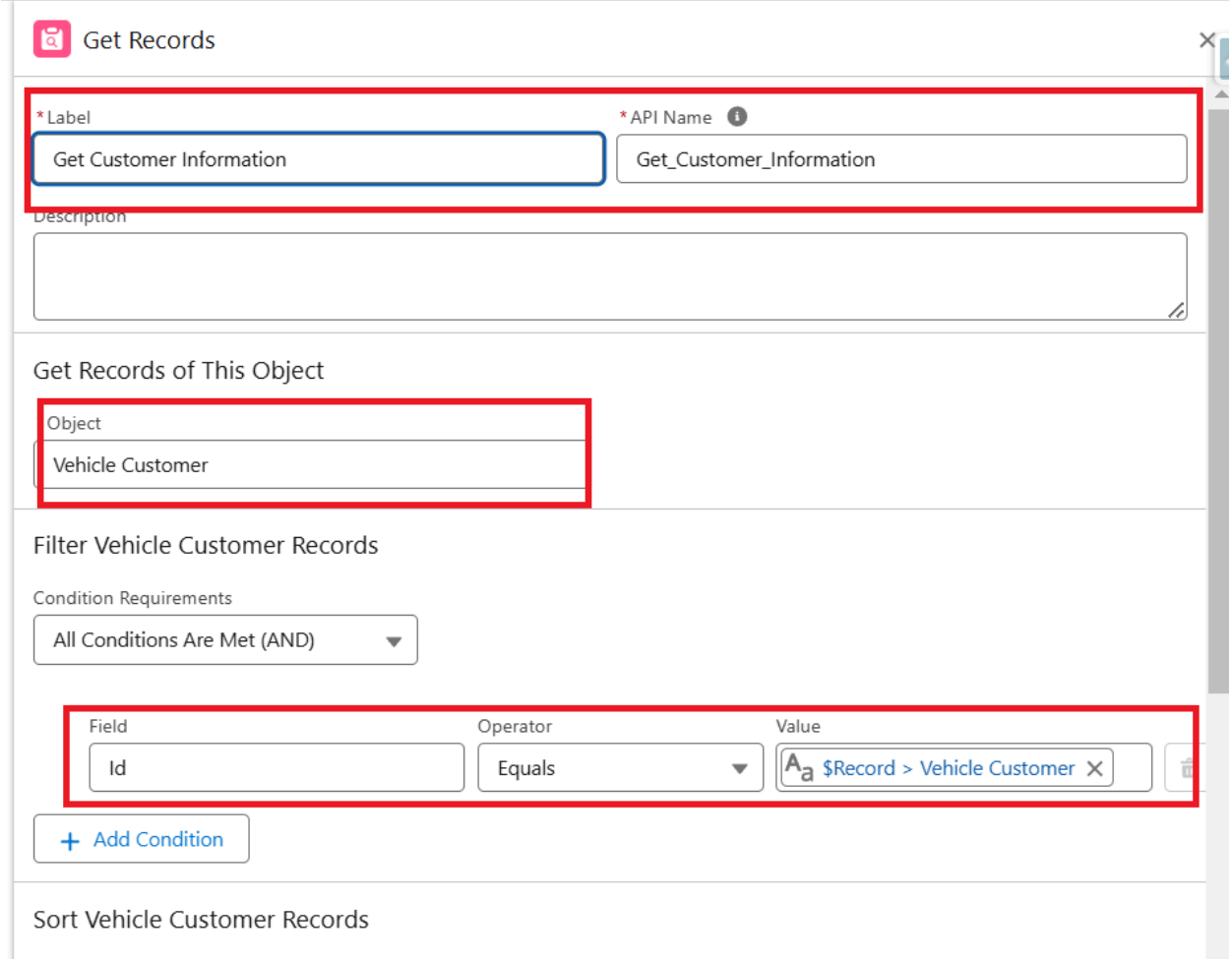
Label: Reminder Before Test Drive.
Time Source: Test_Drive_Date__c
Offset Number: 1.
Offset Options: Days Before.
Click Done.


Conditions: 1
Optimize for: Actions and Related Records
Add Scheduled Paths (Optional)
Open Flow Trigger Explorer for Vehicle Order
Run Immediately


Configure Scheduled Paths
SCHEDULED PATHS
Run Immediately
Reminder Before Test Drive.
Add a scheduled path if you want a flow path to run a certain amount of time after the triggering record is created or updated. That time can be based on the triggering event or on a specified date or date/time field in the record. A separate flow interview runs for each scheduled path. Each scheduled path interview is queued to run at the scheduled time, but can be delayed until system resources become available. Path labels appear on connectors in the canvas.
SCHEDULED PATH DETAILS
Delete Path
* Path Label
Reminder Before Test Drive.
* API Name
Reminder_Before_Test_Drive
Vehicle_Test_Drive__c: Test Drive Date
* Offset Number
1
* Offset Options
Days Before
Show advanced options

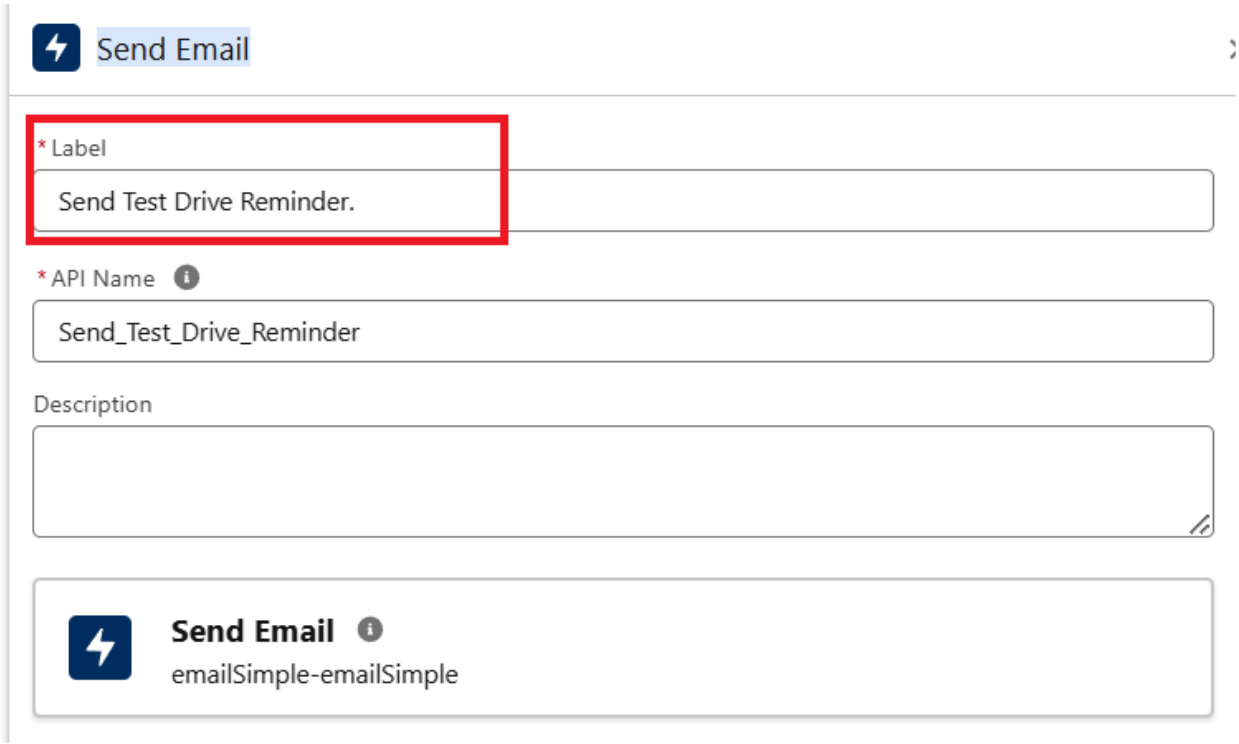
Step 4 :

1. Click + Add Element → Get Records.
2. Label: Get Customer Information.
3. Object: Vehicle_Customer__c.
4. Filter Conditions: Id = {!$Record.Customer__c}.
5. How Many Records to Store: Select Only the first record.
6. How to Store Record Data: Choose Automatically store all fields.

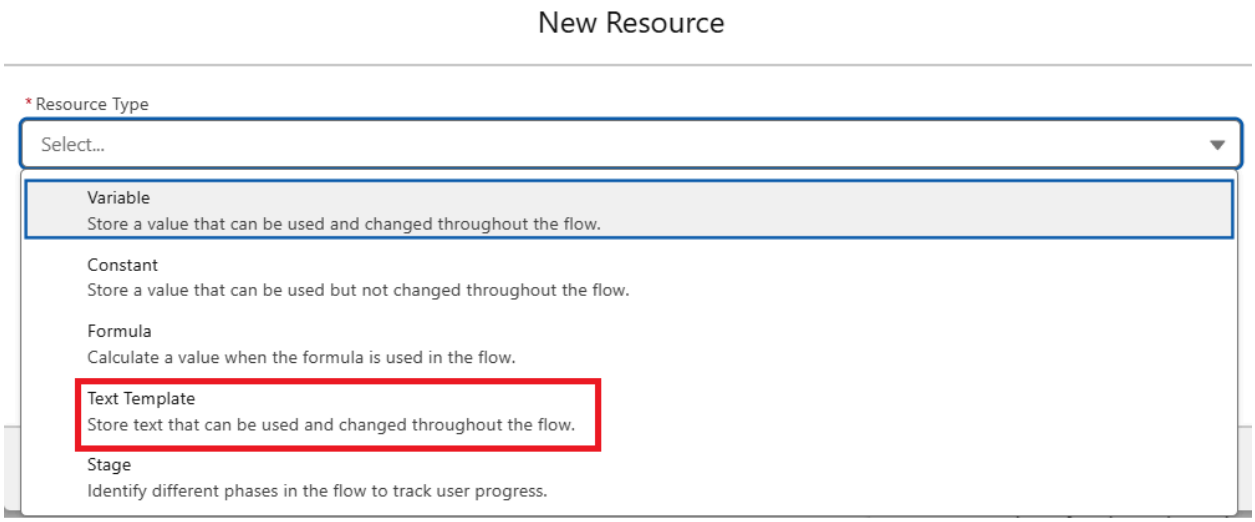


Step 5: Send Reminder Email

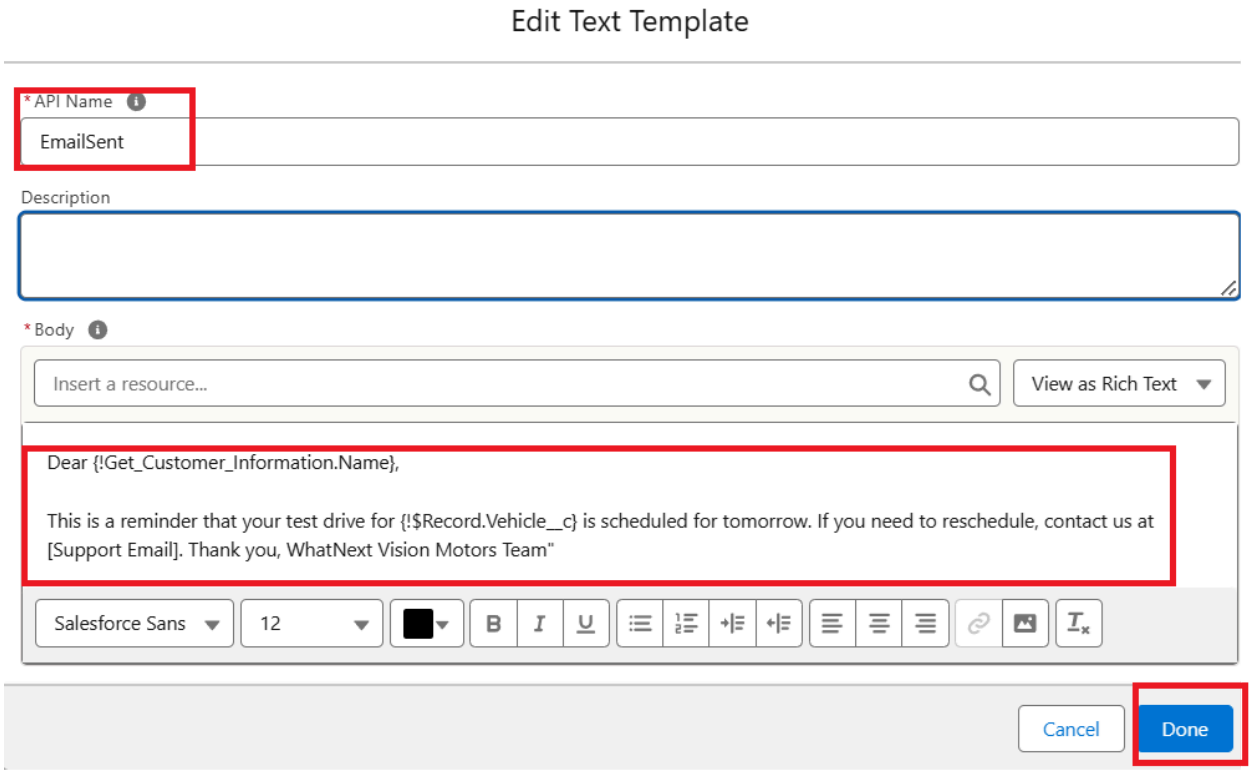
1. Click + Add Element → Action.
2. Action Type: Send Email
3. Label: Send Test Drive Reminder.
4. Subject: "Reminder: Your Test Drive is Tomorrow!".
5. Recipient Address List : {!Get_Customer_Information.Email__c}
6. Rich-Text-Formatted Body : True

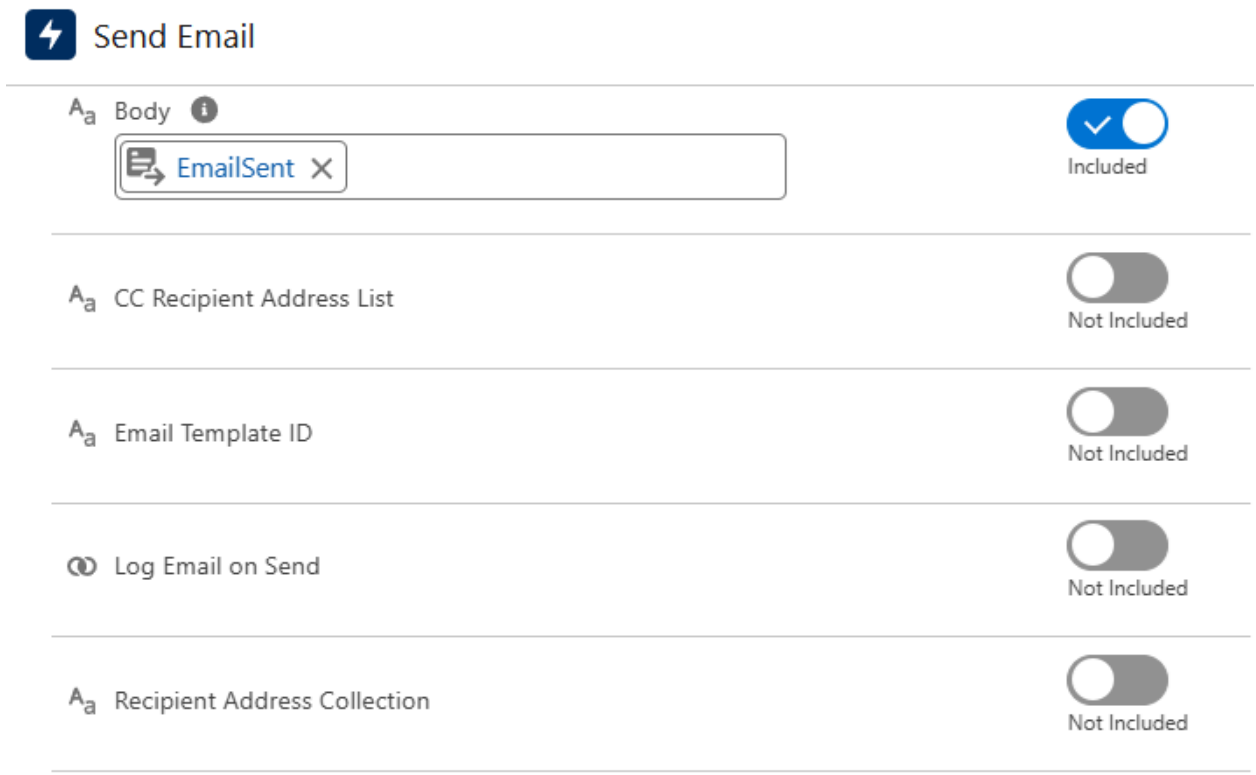


7. Body : Create Variable



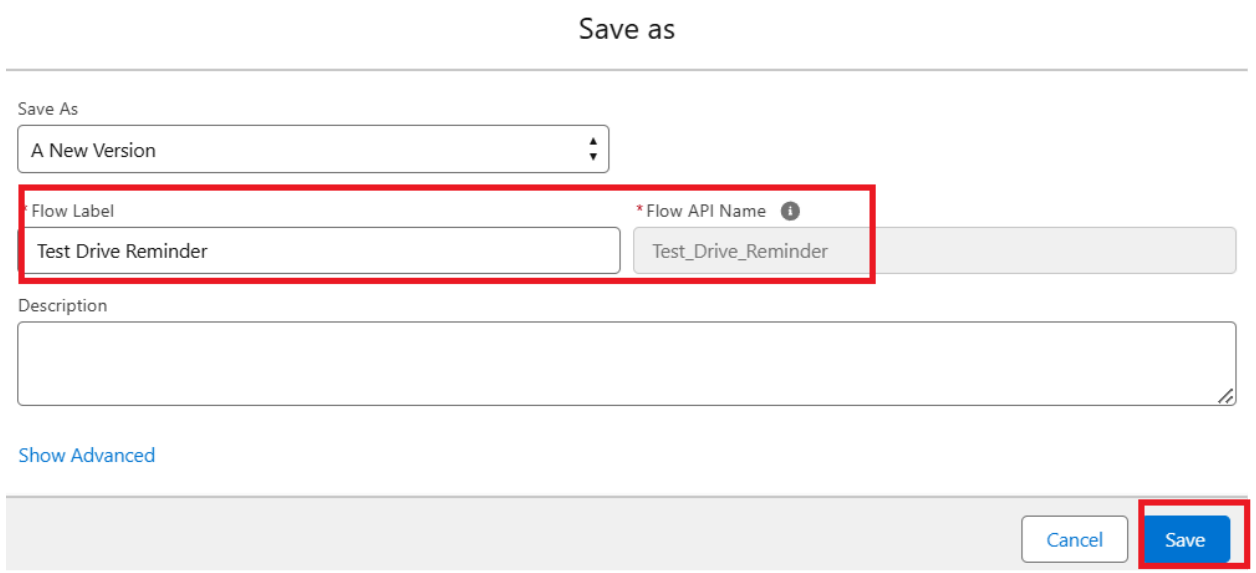
8 . Api Name : EmailSent





Step 6 : Click save

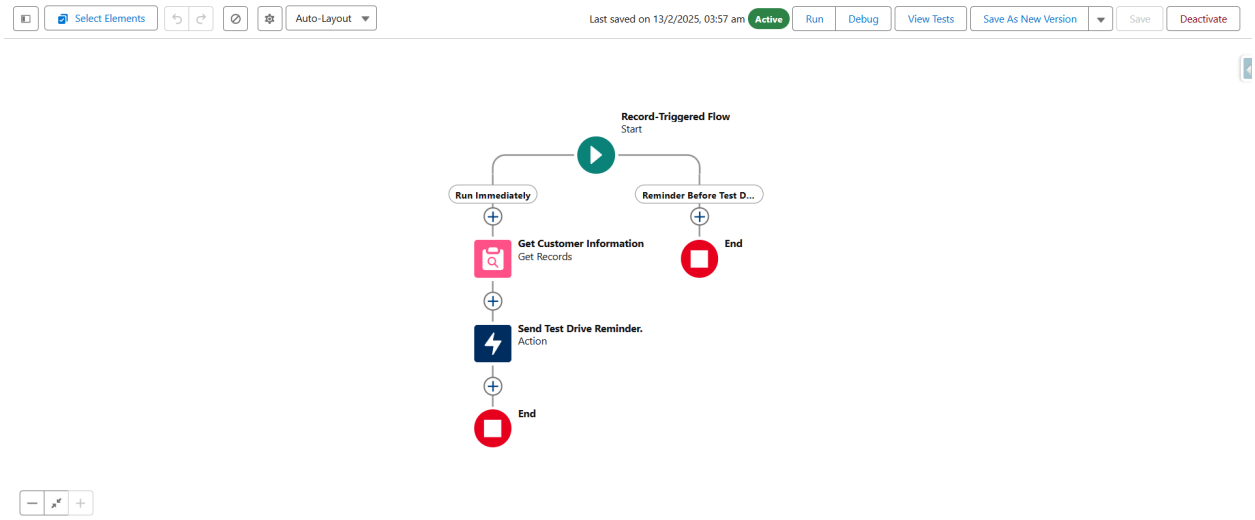
Label Name : Test Drive Reminder



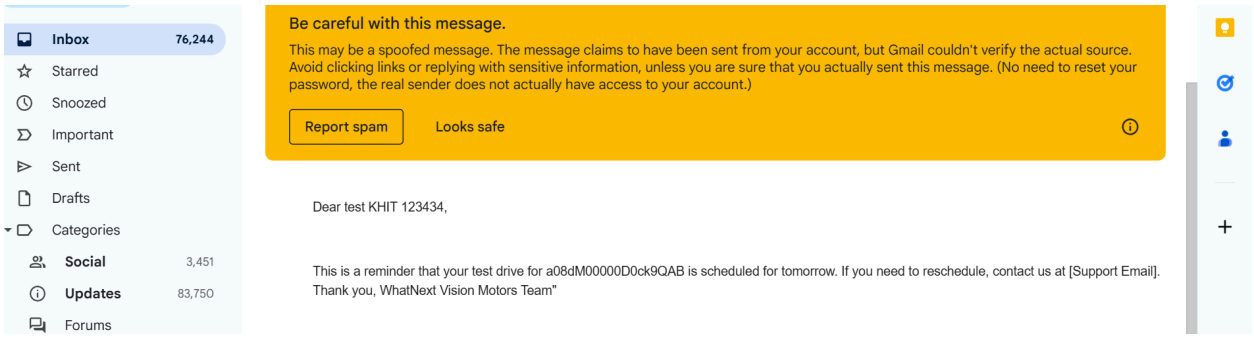
Step 7 : Activate Flow

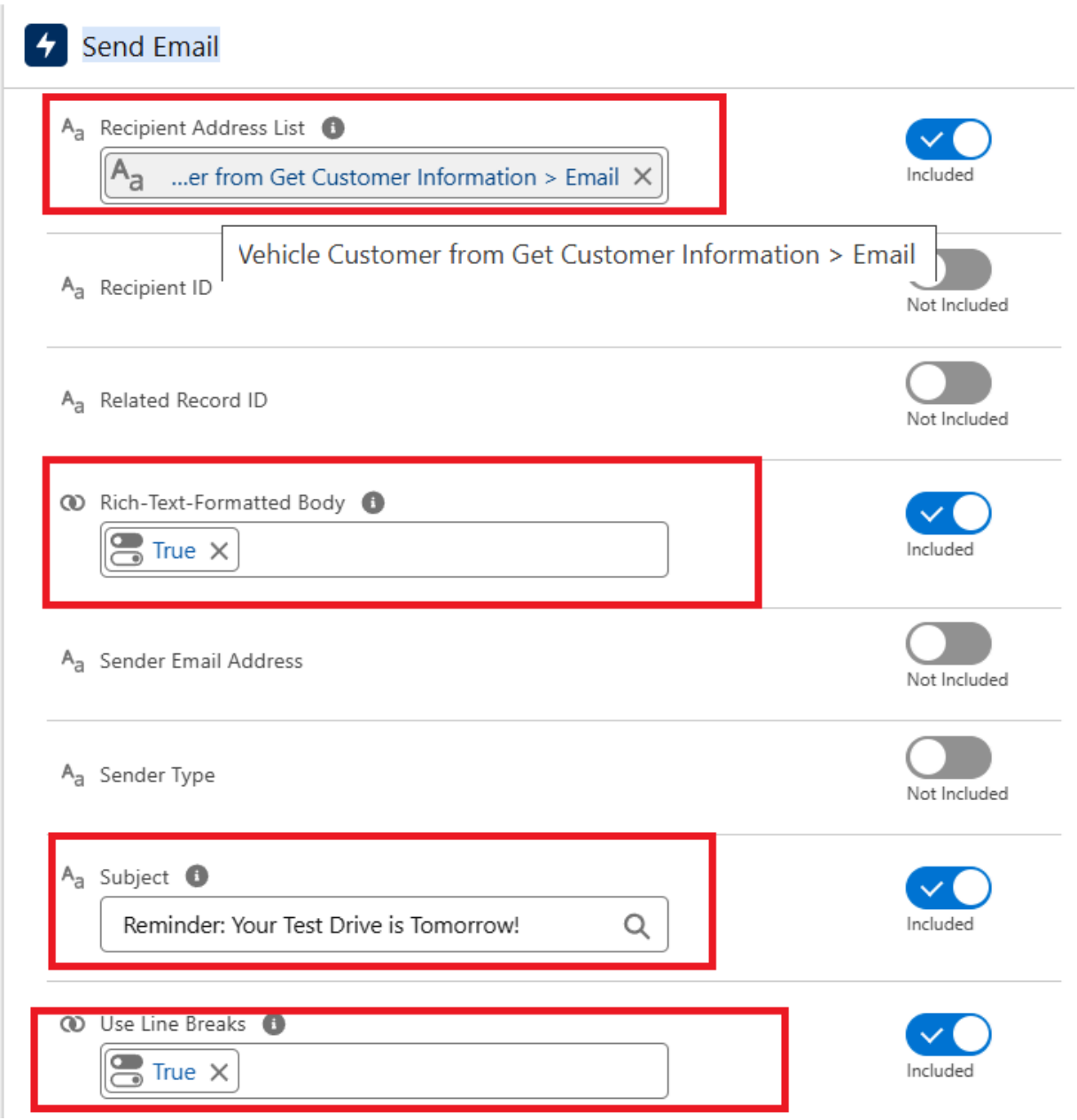


Flow Page After Creation :



Test your Flow :

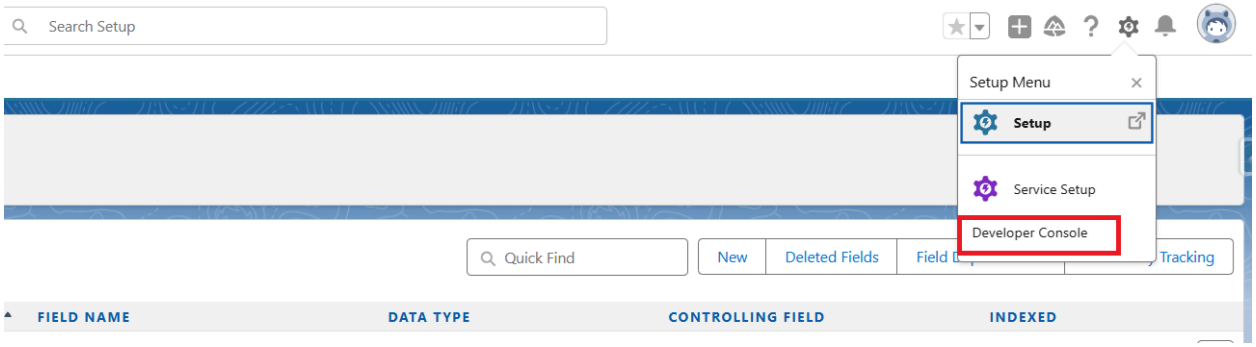



Send Email
Recipient Address List
...er from Get Customer Information > Email
Included
Recipient ID
Vehicle Customer from Get Customer Information > Email
Not Included
Related Record ID
Not Included
Rich-Text-Formatted Body
True
Included
Sender Email Address
Not Included
Sender Type
Not Included
Subject
Reminder: Your Test Drive is Tomorrow!
Included
Use Line Breaks
True
Included

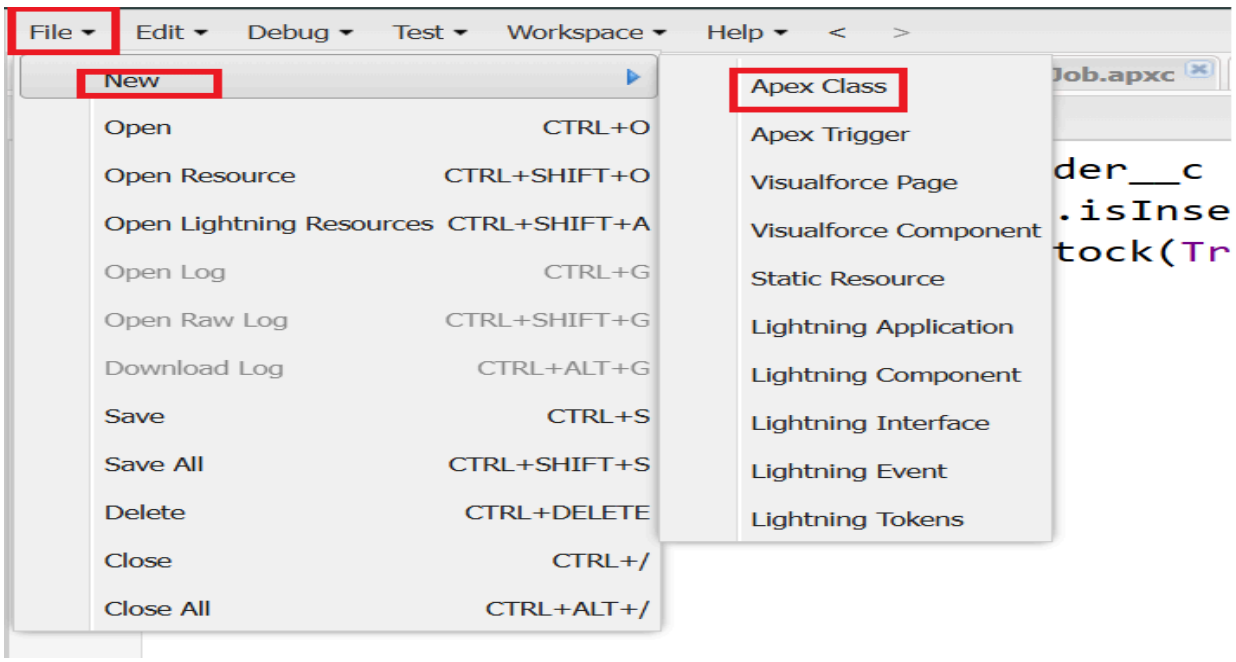
MileStone 7 :

Create Apex and Trigger Batch Jobs :

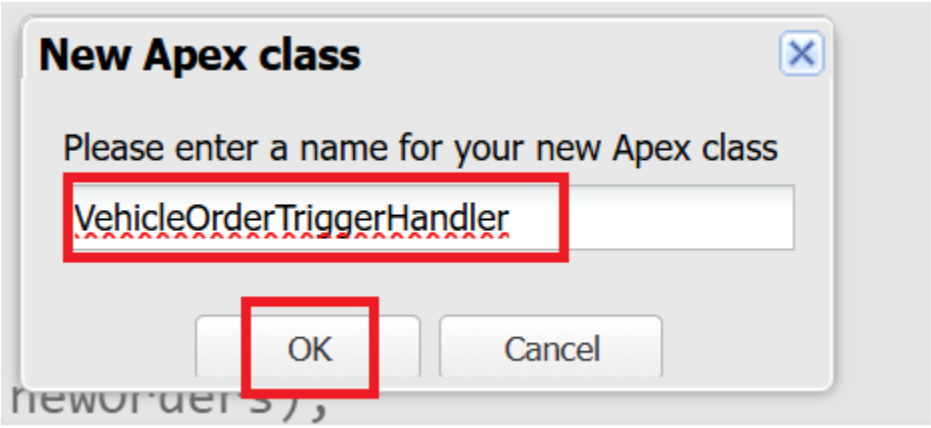
Step 1 : Click Developer Console From Gear icon



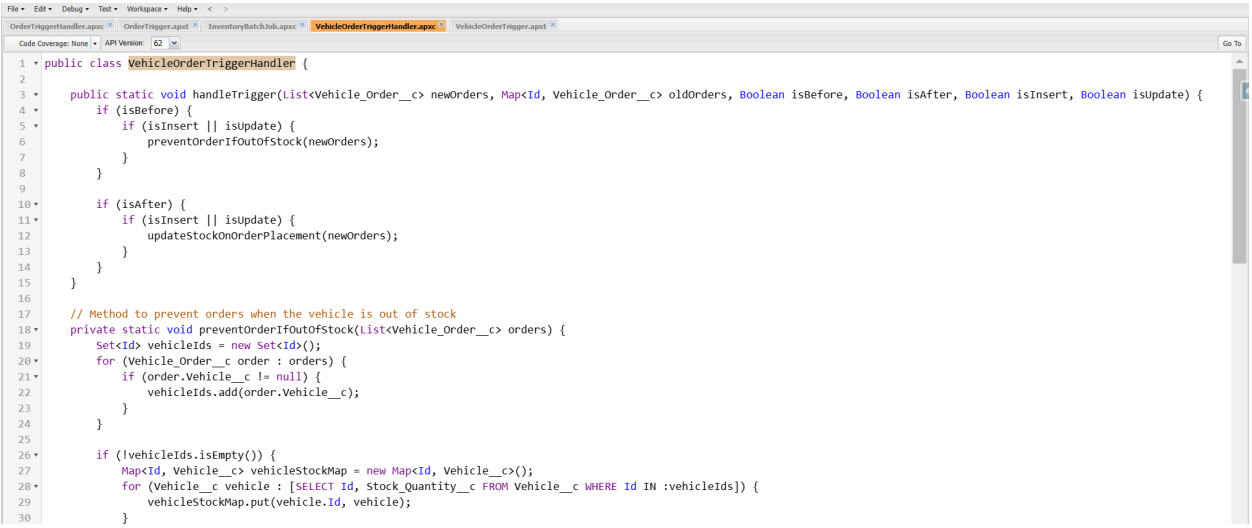
Step 2 : click File then Select New then Select Apex.



Step 3 : Give Name For Apex Class
Class Name : VehicleOrderTriggerhandler



Step 4 : Write Apex Code



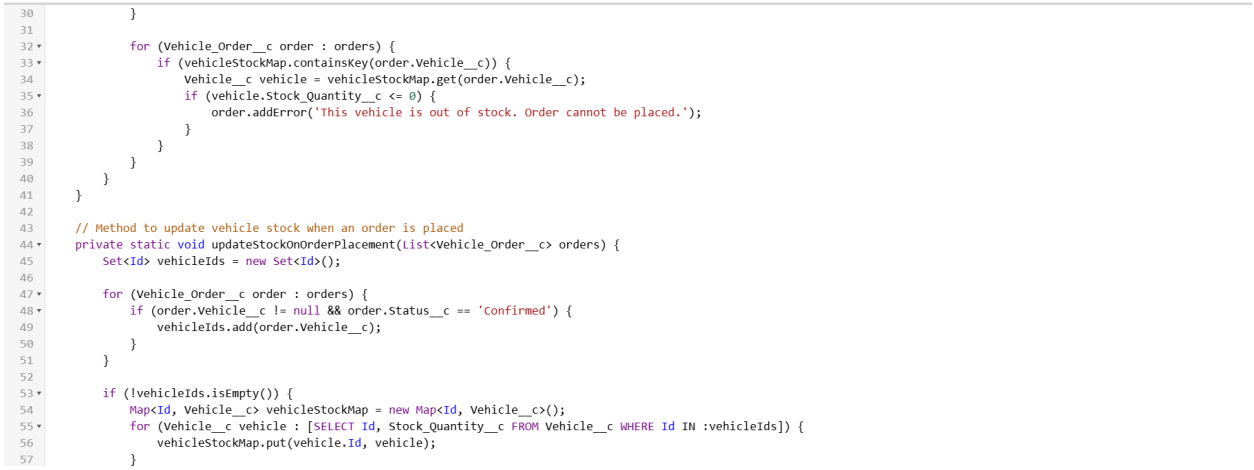
```
public class VehicleOrderTriggerHandler {

    public static void handleTrigger(List<Vehicle_Order__c> newOrders, Map<Id, Vehicle_Order__c> oldOrders, Boolean isBefore, Boolean isAfter, Boolean isInsert, Boolean isUpdate) {
        if (isBefore) {
            if (isInsert || isUpdate) {
                preventOrderIfOutOfStock(newOrders);
            }
        }

        if (isAfter) {
            if (isInsert || isUpdate) {
                updateStockOnOrderPlacement(newOrders);
            }
        }
    }

    // Method to prevent orders when the vehicle is out of stock
    private static void preventOrderIfOutOfStock(List<Vehicle_Order__c> orders) {
        Set<Id> vehicleIds = new Set<Id>();
        for (Vehicle_Order__c order : orders) {
            if (order.Vehicle__c != null) {
                vehicleIds.add(order.Vehicle__c);
            }
        }

        if (!vehicleIds.isEmpty()) {
            Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
            for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE Id IN :vehicleIds]) {
                vehicleStockMap.put(vehicle.Id, vehicle);
            }
```



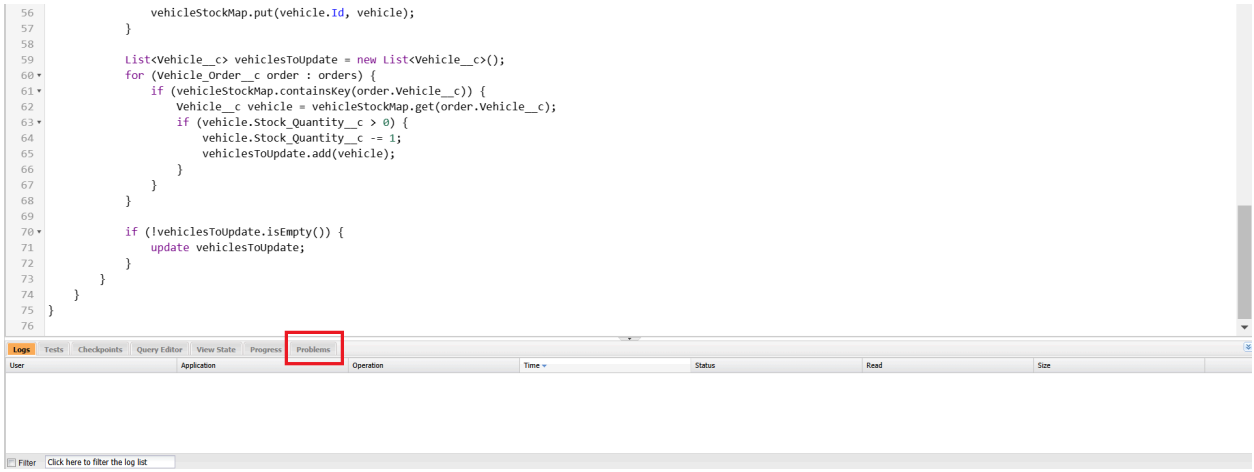
```
            }

            for (Vehicle_Order__c order : orders) {
                if (vehicleStockMap.containsKey(order.Vehicle__c)) {
                    Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
                    if (vehicle.Stock_Quantity__c <= 0) {
                        order.addError('This vehicle is out of stock. Order cannot be placed.');
                    }
                }
            }
        }
    }

    // Method to update vehicle stock when an order is placed
    private static void updateStockOnOrderPlacement(List<Vehicle_Order__c> orders) {
        Set<Id> vehicleIds = new Set<Id>();

        for (Vehicle_Order__c order : orders) {
            if (order.Vehicle__c != null && order.Status__c == 'Confirmed') {
                vehicleIds.add(order.Vehicle__c);
            }
        }

        if (!vehicleIds.isEmpty()) {
            Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
            for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE Id IN :vehicleIds]) {
                vehicleStockMap.put(vehicle.Id, vehicle);
            }
```



```
                vehicleStockMap.put(vehicle.Id, vehicle);
            }

            List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();
            for (Vehicle_Order__c order : orders) {
                if (vehicleStockMap.containsKey(order.Vehicle__c)) {
                    Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
                    if (vehicle.Stock_Quantity__c > 0) {
                        vehicle.Stock_Quantity__c -= 1;
                        vehiclesToUpdate.add(vehicle);
                    }
                }
            }

            if (!vehiclesToUpdate.isEmpty()) {
                update vehiclesToUpdate;
            }
        }
    }
}
```

Logs | Tests | Checkpoints | Query Editor | View State | Progress | Problems

User	Application	Operation	Time	Status	Read	Size

Filter | Click here to filter the log list

Source Code :

```
public class VehicleOrderTriggerHandler {

   public static void handleTrigger(List<Vehicle_Order__c> newOrders, Map<Id,
Vehicle_Order__c> oldOrders, Boolean isBefore, Boolean isAfter, Boolean isInsert, Boolean
isUpdate) {
      if (isBefore) {
         if (isInsert || isUpdate) {
            preventOrderIfOutOfStock(newOrders);
         }
      }

      if (isAfter) {
         if (isInsert || isUpdate) {
            updateStockOnOrderPlacement(newOrders);
```

```
        }
    }
}

// Method to prevent orders when the vehicle is out of stock
private static void preventOrderIfOutOfStock(List<Vehicle_Order__c> orders) {
    Set<Id> vehicleIds = new Set<Id>();
    for (Vehicle_Order__c order : orders) {
        if (order.Vehicle__c != null) {
            vehicleIds.add(order.Vehicle__c);
        }
    }

    if (!vehicleIds.isEmpty()) {
        Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
        for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE
Id IN :vehicleIds]) {
            vehicleStockMap.put(vehicle.Id, vehicle);
        }

        for (Vehicle_Order__c order : orders) {
            if (vehicleStockMap.containsKey(order.Vehicle__c)) {
                Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
                if (vehicle.Stock_Quantity__c <= 0) {
                    order.addError('This vehicle is out of stock. Order cannot be placed.');
                }
            }
        }
    }
}

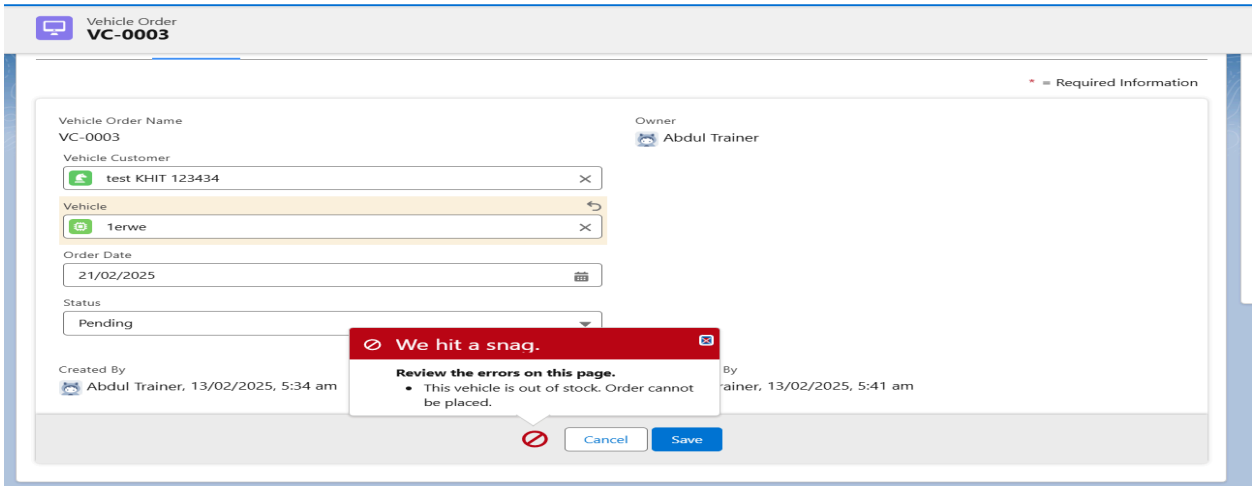
// Method to update vehicle stock when an order is placed
private static void updateStockOnOrderPlacement(List<Vehicle_Order__c> orders) {
    Set<Id> vehicleIds = new Set<Id>();

    for (Vehicle_Order__c order : orders) {
        if (order.Vehicle__c != null && order.Status__c == 'Confirmed') {
            vehicleIds.add(order.Vehicle__c);
        }
    }
```

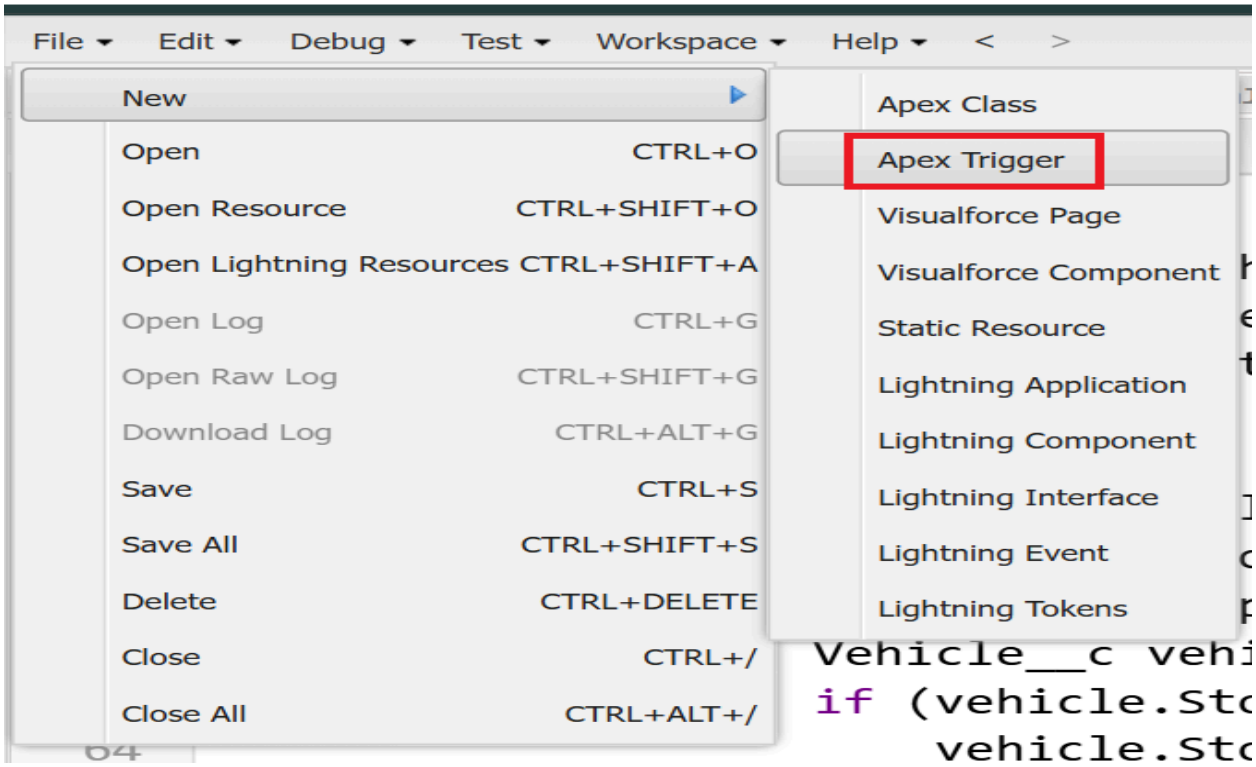
```
        if (!vehicleIds.isEmpty()) {
            Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
            for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE
Id IN :vehicleIds]) {
                vehicleStockMap.put(vehicle.Id, vehicle);
            }

            List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();
            for (Vehicle_Order__c order : orders) {
                if (vehicleStockMap.containsKey(order.Vehicle__c)) {
                    Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
                    if (vehicle.Stock_Quantity__c > 0) {
                        vehicle.Stock_Quantity__c -= 1;
                        vehiclesToUpdate.add(vehicle);
                    }
                }
            }

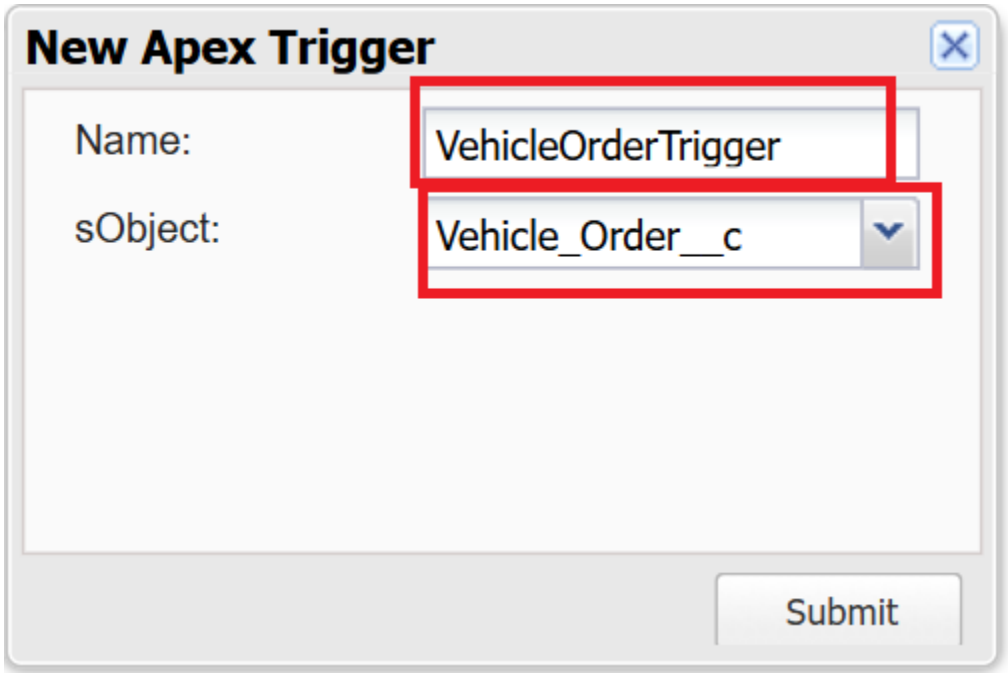
            if (!vehiclesToUpdate.isEmpty()) {
                update vehiclesToUpdate;
            }
        }
    }
}
```



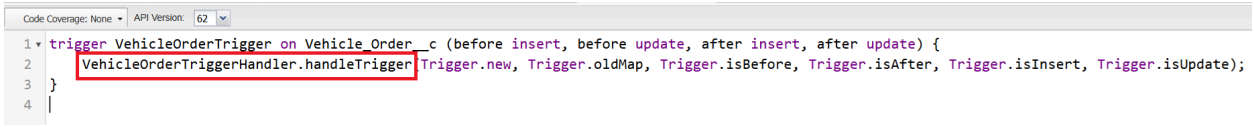
Step 5 : Write Trigger Handler



Step 6 : Writer Trigger Class Name and Select Vehicle Order Object



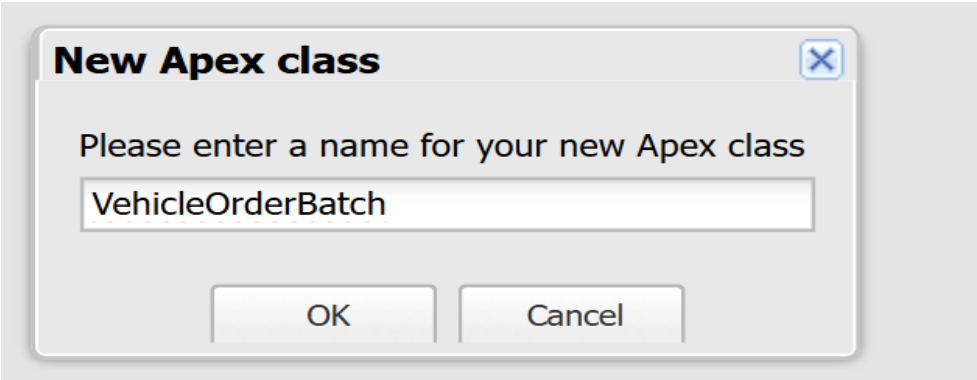
Step 7 : Call Apex Class in Trigger Class



Step 8 : Create Batch Job

A customer places an order, but the vehicle is out of stock.

- The order remains pending.
- After new stock is added, the batch job updates the order to confirmed.



Code Coverage: None API Version: 62

```apex
global class VehicleOrderBatch implements Database.Batchable<sObject> {

    global Database.QueryLocator start(Database.BatchableContext bc) {
        return Database.getQueryLocator([
            SELECT Id, Status__c, Vehicle__c
            FROM Vehicle_Order__c
            WHERE Status__c = 'Pending'
        ]);
    }

    global void execute(Database.BatchableContext bc, List<Vehicle_Order__c> orderList) {
        Set<Id> vehicleIds = new Set<Id>();

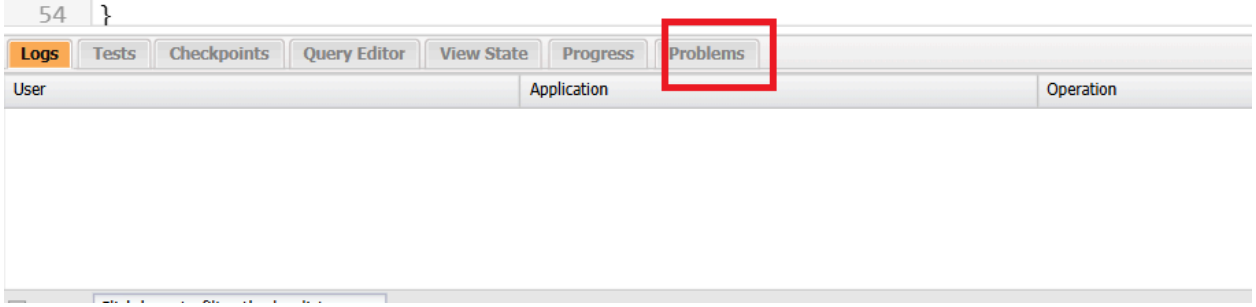
        for (Vehicle_Order__c order : orderList) {
            if (order.Vehicle__c != null) {
                vehicleIds.add(order.Vehicle__c);
            }
        }

        if (!vehicleIds.isEmpty()) {
            Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
            for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE Id IN :vehicleIds]) {
                vehicleStockMap.put(vehicle.Id, vehicle);
            }

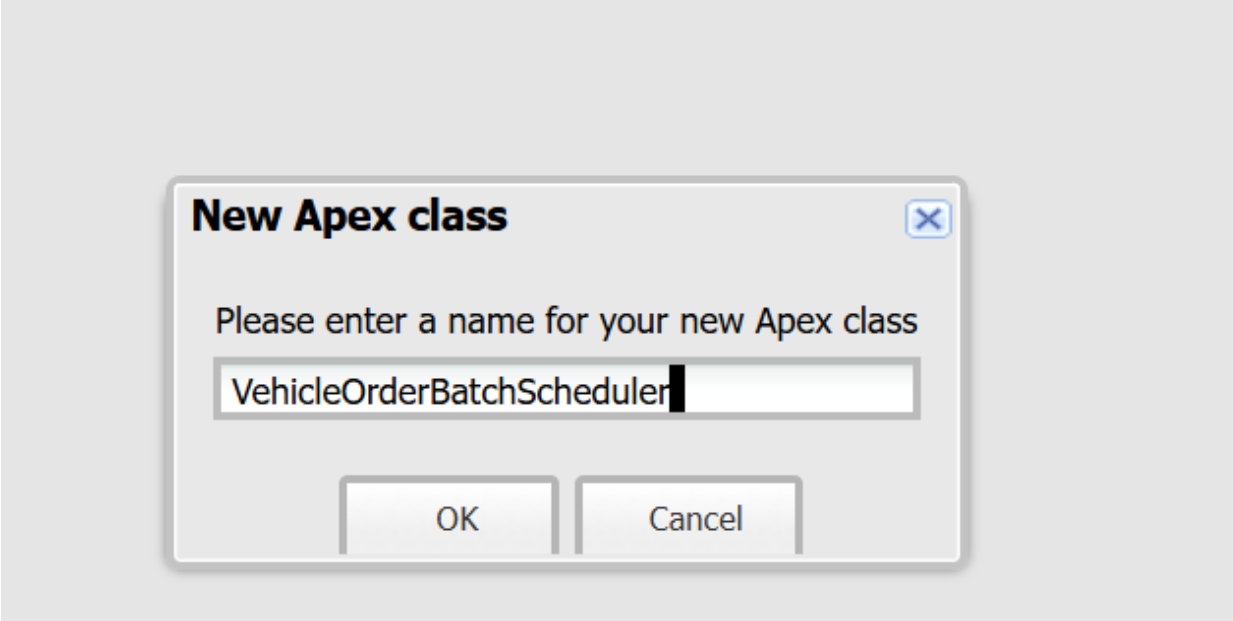
            List<Vehicle_Order__c> ordersToUpdate = new List<Vehicle_Order__c>();
            List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();

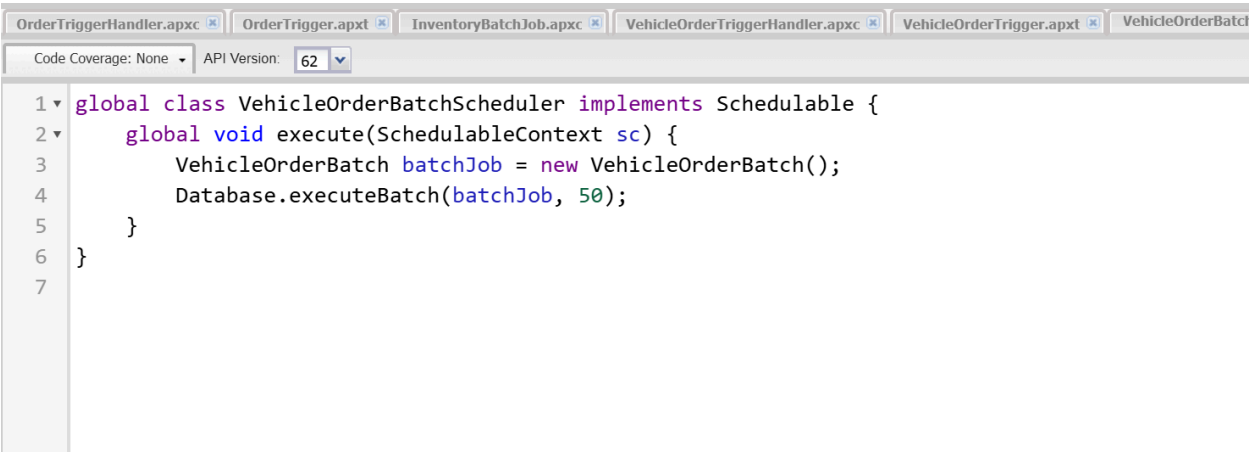
            for (Vehicle_Order__c order : orderList) {
                if (vehicleStockMap.containsKey(order.Vehicle__c)) {
```

```
30                 if (vehicleStockMap.containsKey(order.Vehicle__c)) {
31                     Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
32                     if (vehicle.Stock_Quantity__c > 0) {
33                         order.Status__c = 'Confirmed';
34                         vehicle.Stock_Quantity__c -= 1;
35                         ordersToUpdate.add(order);
36                         vehiclesToUpdate.add(vehicle);
37                     }
38                 }
39             }
40
41             if (!ordersToUpdate.isEmpty()) {
42                 update ordersToUpdate;
43             }
44
45             if (!vehiclesToUpdate.isEmpty()) {
46                 update vehiclesToUpdate;
47             }
48         }
49     }
50
51     global void finish(Database.BatchableContext bc) {
52         System.debug('Vehicle order batch job completed.');
53     }
54 }
```



Step 9 : Create Schedule Class then revoked Batch Class in Schedule Class

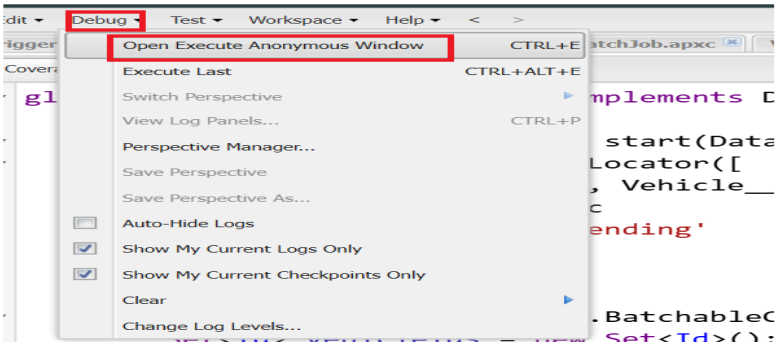


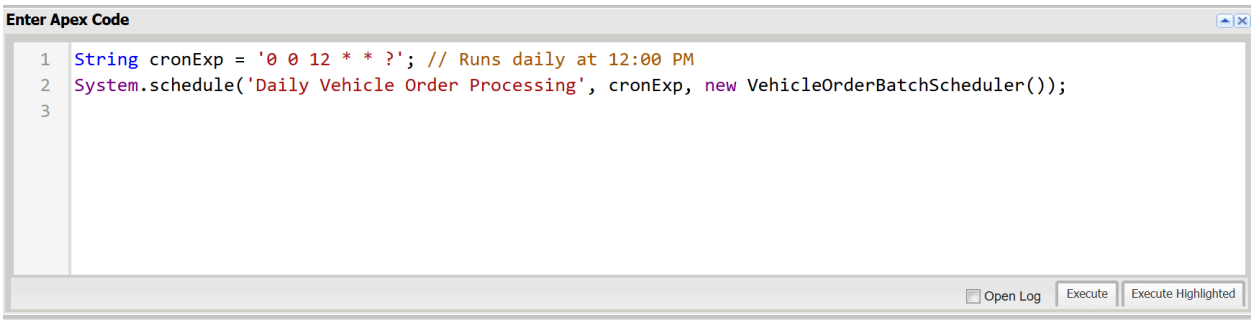


Source Code :

```
global class VehicleOrderBatchScheduler implements Schedulable {
   global void execute(SchedulableContext sc) {
      VehicleOrderBatch batchJob = new VehicleOrderBatch();
      Database.executeBatch(batchJob, 50); // 50 is the batch size
   }
}
```

Step 10 : **Schedule the Batch Job** To run the batch job **every night at midnight** :

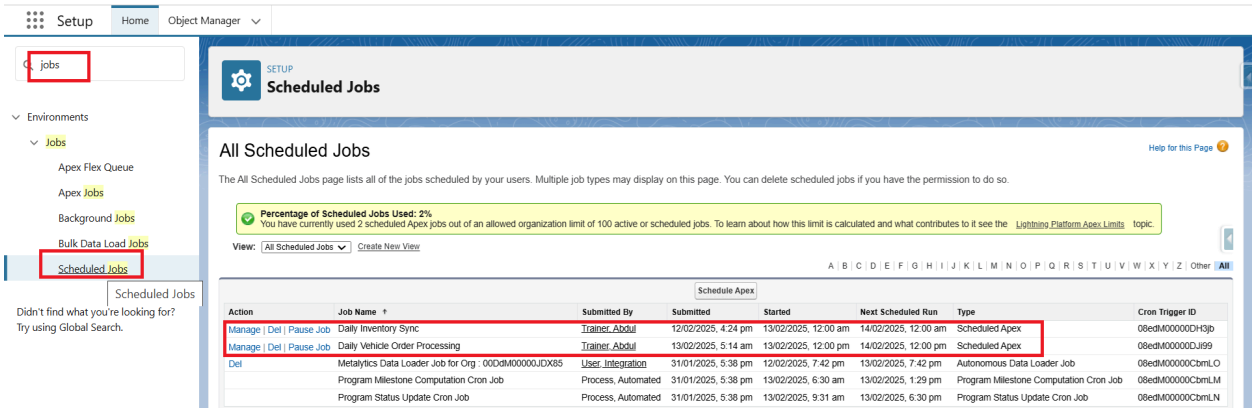




Source Code :
String cronExp = '0 0 12 * * ?'; // Runs daily at 12:00 PM

System.schedule('Daily Vehicle Order Processing', cronExp, new VehicleOrderBatchScheduler());

Step 11 : You Can Check where this Schedule Job is Running



Demo Drive Link:

https://drive.google.com/drive/u/0/folders/1cGsmtglcclFkLsscfpsolGfvlK_StfAB

Thank you